
PEEK: Predictive Queue-Informed KV Cache Management for LLM Serving*

Bing Xie[†] Zhipeng Wang Masahiro Tanaka Zheng Zhen

Abstract

We present PEEK, a lightweight scheduling and eviction framework for both online (streaming) and offline (batch) LLM serving; this paper focuses on the online regime. PEEK maintains an incremental *radix tree over the pending queue*, exposing prefix-sharing clusters no existing engine surfaces. A low-overhead *dual-walk* matches the tree against the engine’s prefix cache to yield longest-prefix-match for every waiting request; PEEK then admits cluster pioneers first so siblings inherit the freshly cached prefix, a co-designed eviction hook protects blocks ancestral to queued demand, and a *multi-lane* stride scheduler bounds starvation. On SGLang and vLLM across five workloads up to $4\times H100$ (DP=2 over TP=2), PEEK delivers up to $3.0\times/2.6\times$ cache hit, $7.9\times/7.1\times$ TTFT, $6.7\times/5.5\times$ E2E, and $3.6\times/4.5\times$ throughput gains over each engine’s strongest stock baseline (SGLang/vLLM), while matching baselines within noise on workloads with no exploitable prefix structure. Wins hold as KV-cache pressure and inference parallelism scale.

1 Introduction

Modern LLM serving engines optimize KV cache reuse: SGLang’s RadixAttention [Zheng et al., 2024a] provides longest-prefix matching over cached blocks; vLLM’s PagedAttention [Kwon et al., 2023] enables block-level sharing via OS-style paging. Yet both stop at the cache—neither reasons about which *waiting* requests share prefixes with each other, leaving the predictive signal in the request queue unexploited (Table 1). SGLang’s prefix-aware policies (**LPM**, **DFS_WEIGHT**) remain bound to current cache state and degrade once the cache is cold; vLLM defaults to FCFS, which under memory pressure devolves into *cache thrashing* as interleaved prefix groups evict one another (§2). In both engines, scheduling and eviction are decoupled, precluding joint optimization—inefficiencies magnified in long-context workloads such as RAG and agentic pipelines.

We present **PEEK**, a lightweight scheduling and eviction framework for both online (streaming) and offline (batch) LLM serving; *this paper focuses on online*. Our motivation is a simple observation: **the waiting queue is itself a structured workload whose prefix-sharing relationships can directly drive KV-cache reuse**. PEEK retains LPM’s prefix-match objective but feeds it richer signals at lower overhead from an incremental radix tree via three mechanisms (Appendix Table 3):

1. **See the queue’s structure for cheap (§3.1)**. PEEK maintains an incremental radix tree over the pending queue—no per-cycle rebuild, $O(D)$ per insert/remove, $O(1)$ per query on scheduling/eviction signals. To compute LPM hits against the engine cache, PEEK’s *dual-walk* co-descends the pending tree against SGLang’s radix cache in one pass, returning all per-request hits in $O(C\times D)$ vs. a naive $O(N\times D)$; for high-sharing workloads $C \ll N$, an N/C amortization³. On vLLM’s flat block-hash cache, PEEK falls back to per-request probes but still reads cluster signals off the tree—structure stock vLLM cannot see.

*Code: <https://github.com/xiexbing/peek>

[†]Corresponding author.

³ N = queue size; C = number of prefix-sharing clusters; D = prompt depth in the tree ($\sim O(\log N)$).

	FCFS (vLLM/SGLang)	LPM (SGLang)	DFS_W (SGLang)	PEEK (Ours)
<i>Scheduling</i>				
Prefix-aware Ordering	✗ Arrival	Cache tree [‡] Per-request	Cache tree [‡] Batch	Queue trie Global
<i>Caching & Eviction</i>				
Mechanism	APC / RadixCache	RadixCache	RadixCache	Pending trie
Eviction	LRU/LFU	LRU/LFU	LRU/LFU	Queue-aware

Table 1: Scheduling and eviction mechanisms in current serving engines. PEEK is the only engine that jointly optimizes scheduling and eviction. [‡]LPM/DFS_W see only cache state, not queue sharing.

2. **Cluster-aware admit and evict (§3.2).** PEEK’s *Cluster-LPM* (cLPM) extends LPM with pending-tree signals on top of its hit-length objective h . Requests in the same cluster therefore rank close together, so one prefill amortizes across the cluster before unrelated arrivals can evict the prefix—instead of scattering by arrival order and thrashing the cache (e.g., stock LPM on hit-length ties; §2). The eviction hook reads the same tree from the other side: zero-demand blocks evict first; blocks the largest pending clusters depend on are protected last.
3. **Bound starvation, adaptively (§3.2).** We observed that cluster-first ordering can starve singletons. PEEK adds a fairness lane keyed by arrival, stride-interleaved with cLPM, and a *dynamic-lane* controller widens the fairness share as singletons accumulate or as the oldest singleton’s wait nears the SLO (EMA-smoothed). Thus, cluster locality dominates on prefix-rich queues; fairness ramps up only when the queue tilts singleton-heavy.

We evaluate PEEK on five workloads spanning single-GPU and multi-GPU deployments (up to $4 \times \text{H100}$, DP=2 over TP=2) across SGLang and vLLM (Appendix Table 6). Our key findings:

- **Substantial wins where prefix structure exists (§4.1–§4.3).** Across shared-prompt chat (W1), long-prefix RAG (W2), and multi-GPU 70B variants of W1 and W2 (W3; $4 \times \text{H100}$, DP=2 $\times$ TP=2), PEEK lifts cache hit $3.0 \times / 2.6 \times$, TTFT $7.9 \times / 7.1 \times$, E2E $6.7 \times / 5.5 \times$, and throughput $3.6 \times / 4.5 \times$ (SGLang/vLLM); gains scale with KV pressure (2–16 $\times$) and parallelism.
- **No regression on low-headroom traffic (§4.4).** On agentic conversation bursts already prefix-coherent by arrival (W4) and singleton-dominated chat (W5), PEEK matches the strongest stock baselines within noise on E2E and throughput ($\pm 2\%$ on W4, $\pm 2\%$ on W5; TTFT envelopes are wider, ± 5 –7%) and modestly wins where weak overlap exists (W5 cell_C_long: +10 pp cache hit, up to $2.6 \times$ TTFT on vLLM)—never regressing.
- **Cluster-aware admission is the dominant lever; other components refine it (§4.1).** cLPM alone lands within **3 pp** cache hit of the full stack. Group-major (GM) shifts cache hit by **−0.1 to +1.4 pp** (SGLang) / **−3.0 to −0.4 pp** (vLLM) but costs **10–29%** TTFT/E2E from singleton deferral; dynamic-lane (DL) fairness recovers **2.4–15.1%**; queue-aware eviction (PE) further trims TTFT/E2E by **0–3%** on SGLang, flat on vLLM.

PEEK also supports the *offline* (batch) regime, resting on the same observation: the pending queue makes future cache demand *predictive*, and the prefix relationships among waiting requests are the highest-signal structure to schedule and evict against. In contrast, PEEK-offline builds a one-shot prefix trie over the full submitted batch, DFS-traverses it to place sharing requests adjacently from a cold cache, and refines wave-level ordering against a shadow LRU model with the same queue-aware eviction hooks (Appendix A.3.1). PEEK-offline delivers up to $1.6 \times / 1.8 \times$ throughput on single-GPU (Qwen2.5-14B/1 $\times$ H100) and $2.4 \times / 8.0 \times$ on multi-GPU (Qwen2.5-72B/2 $\times$ H100) over the strongest baselines (SGLang/vLLM), with near-zero overhead on non-sharing workloads (Appendix A.3.2).

2 Background and Motivation

Prefix caching mechanisms. We focus on SGLang and vLLM, the leading open-source LLM serving engines in production use; a *request* is a single completion call (prefill over the input prompt followed by autoregressive decoding). Both engines use *PagedAttention* to decouple logical token sequences from physical KV cache blocks. On top of this, vLLM employs APC (Automatic Prefix Caching), which hashes each block by token content and prefix position via chained hashing to identify reusable blocks. SGLang adopts **RadixCache**, organizing cached blocks in a radix tree for $\mathcal{O}(D)$ per-request LPM. These mechanisms provide the *architectural possibility* for prefix sharing; actual performance depends on the *scheduling* and *eviction* policies discussed below.

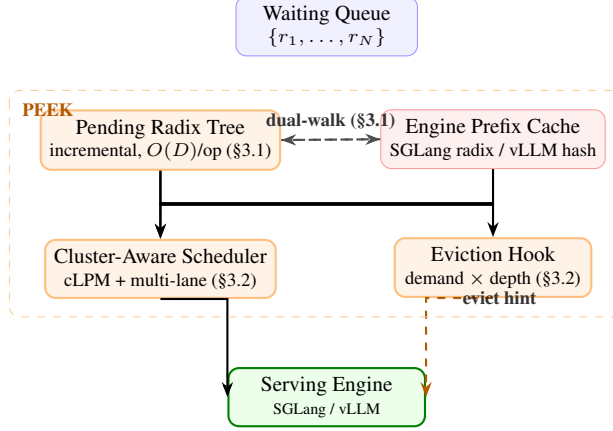


Figure 1: PEEK architecture.

Scheduling and eviction policies. vLLM uses **FCFS** (arrival order, $\mathcal{O}(1)$, prefix-agnostic); SGLang offers **LPM** (Longest Prefix Match, $\mathcal{O}(N \cdot D)$ per round) and **DFS_WEIGHT**, which groups requests by cache-tree position. All three bind to the *current* cache state and ignore inter-queue sharing: FCFS+APC barely helps under cache pressure (§4.1); LPM (greedy per-round) and DFS_WEIGHT (cache-tree-position grouping) both decide from current cache state and miss waiting-queue sharing. On the eviction side, both engines default to **LRU**, designed for OS page caches with unknown future access—but the LLM waiting queue encodes upcoming work, and LRU may evict prefixes that pending requests need next. SGLang’s **LFU** alternative is similarly queue-oblivious. Under heavy contention, all of these struggle (§4.1).

Scheduling-induced thrashing. Stock LPM ranks by hit length h and breaks ties by arrival—reactive, blind to future queue demand. Consider 8 cold chat requests interleaved across three tenants ($A_1, B_1, C_1, A_2, B_2, C_2, A_3, B_3$; cluster sizes 3/3/2; each tenant’s $\sim 2K$ -token prompt uncached) on a KV cache that holds only one prompt at a time. All tied at $h=0$, LPM admits in arrival order: prefill A , evict for B , evict for C , refill A —**0% hit**. PEEK reorders the queue by cluster via the pending tree to $A_1, A_2, A_3, B_1, B_2, B_3, C_1, C_2$: each prompt prefills once and is reused—**62.5% hit**. At 100+ tenants and $2\text{--}16\times$ KV pressure, the gap widens: cache-state-bound schedulers cannot anticipate which prefix the queue will demand next (§4.1).

The gaps. (i) *Visibility*: existing schedulers ignore inter-queue sharing. (ii) *Coupling*: scheduling and eviction are decoupled. (iii) *Co-design*: scheduling and eviction share the same queue-demand signal but no engine exploits it jointly. PEEK closes all three: an incremental radix tree over the waiting queue exposes prefix-sharing clusters the engines cannot see, driving cluster-aware admission and a co-designed eviction hook that protects blocks queued requests will reuse (§3).

3 PEEK: Predictive Queue-Informed KV Cache Management

PEEK rests on a simple observation: **the waiting queue is itself a structured workload**. To utilize its signals, PEEK introduces an incremental radix tree over the pending queue (§3.1) supporting three mechanisms (Figure 1): **(1)** a *dual-walk* (§3.1) yielding all-LPM via $\mathcal{O}(C \times D)$ tree co-descent on SGLang’s radix cache and per-request hash probes on vLLM’s hash cache; **(2)** *cluster-aware admission* (§3.2) admits the pioneer first so siblings reuse the cached prefix, paired with an *eviction hook* that frees only blocks that no pending cluster needs; **(3)** a *multi-lane stride scheduler* (§3.2) interleaving cache-locality with fairness.

PEEK pairs a Rust core (~ 1.6 kLOC; pending tree, dual-walk, cluster/score lookups; PyO3) with ~ 800 LOC Python shims per engine that monkey-patch SGLang’s RadixCache/Scheduler and vLLM v1’s Scheduler/BlockPool; cLPM sort runs in Python over Rust-computed signals. Environment flags toggle PEEK per-binary, no upstream fork (Appendix A.1.2).

Algorithm 1 Dual-Walk on SGLang: per-cycle LPM via tree-vs-tree co-descent

Require: Pending tree T ; SGLang radix cache $\mathcal{C}$

Ensure: LPM hit length $H[r]$ for every waiting request r

```
1:  $H \leftarrow \{\}$ ; VISIT( $T.\text{root}$ , 0); return  $H$ 
2: procedure VISIT(pending node  $u$ , matched depth  $d$ ):
3:   for each child edge  $e = (u \rightarrow u')$  of  $u$  do
4:      $\delta \leftarrow \text{MATCHINRADIX}(e.\text{tokens}, \mathcal{C}, d)$  {co-traverse  $\mathcal{C}$  from depth  $d$ }
5:      $h \leftarrow d + \delta$ 
6:     for each terminator  $r$  in  $\text{SUBTREE}(u')$  do
7:        $H[r] \leftarrow \max(H[r], h)$  {LPM along this branch}
8:     end for
9:     if  $\delta = |e.\text{tokens}|$  then
10:      VISIT( $u'$ ,  $h$ ) {full edge matched: descend}
11:    end if
12:  end for
13: vLLM variant. The flat block-hash cache permits no co-descent. Instead, PEEK calls the engine's per-request find_longest_cache_hit for each  $r$  to obtain  $H[r]$  in  $O(N \times b)$  ( $b$ =block-hash chain length), then walks the pending tree separately ( $O(N)$ ) to read cluster signals.
```

3.1 Pending Radix Tree and Dual-Walk for LPM

The pending radix tree is a compressed trie over the waiting prompts. Each leaf carries the request IDs (*terminators*) whose prompt ends there; each internal node holds a `pending_count`—the number of waiting prompts passing through. Edges store multi-token runs via radix compression, so size scales with distinct prefix branches, not raw token count. PEEK maintains the tree incrementally: insert on arrival, remove on admission/completion, both $O(D)$ along the prompt's path (typically $O(\log N)$). The tree is always current at scheduling time—no per-cycle rebuild. Two requests share a queued prefix iff their prompts pass through a common internal node with `pending_count` ≥ 2 ; the deepest such node is the request's *cluster*, and its `pending_count` the cluster size.

Each cycle, PEEK obtains the LPM hit $h(r)$ for every waiting request and separately reads its cluster signals (size, ancestor score) from the pending tree at $O(1)$ per request. On SGLang's radix cache, PEEK *dual-walks*—tree-vs-tree co-descent of the pending tree against the radix cache (Algorithm 1; Figure 6 in Appendix A.1.4)—collapsing N per-request lookups into C per-cluster lookups, $O(C \times D)$ vs. $O(N \times D)$, an amortization of N/C that is large for high-sharing workloads where $C \ll N$. On vLLM's flat block-hash cache, no structural co-descent is possible: PEEK calls the engine's per-request `find_longest_cache_hit` once per request ($O(N \times b)$, b = block-hash chain length), then reads cluster signals separately from the pending tree—an $O(N)$ traversal exposing the cluster structure stock vLLM cannot see.

3.2 Pending-Tree-Driven Scheduling and Eviction

PEEK's scheduler (cLPM) and eviction hook (PE) both read the same pending tree, so admission and protection are consistent without explicit coordination. The full per-cycle procedure is given in Algorithm 2 (Appendix A.1.5); we walk through its components below.

For every waiting request r , cLPM produces three signals: (1) the *LPM hit* $h(r)$ against the engine cache; (2) the *section* r belongs to—*warm* if $h(r) > \tau$, otherwise *pioneer* if r 's first $\theta=32$ tokens have not yet been claimed this cycle, else *sibling*; (3) two pending-tree statistics, $|\text{cluster}(r)|$ (pending count at r 's deepest ≥ 2 ancestor) and the *ancestor score* $\text{score}(r) = \sum_{v \in \pi(r)} \text{pending_count}(v) \cdot |\text{edge}(v)|$ (depth-weighted demand along r 's path). cLPM then sorts by

$$\text{key}(r) = (\text{section}(r), -h(r), -\text{score}(r), -|\text{cluster}(r)|, \text{arrival}(r)). \quad (1)$$

By comparison, stock LPM is a special case: it sorts by $(-h(r), \text{is_depr}(r), \text{arrival}(r))$, where `is_depr` is the same θ -prefix pioneer/sibling flag. cLPM differs in two ways: it *promotes* the section distinction from a binary tiebreak to the primary key (so any pioneer admits before any sibling, regardless of h), and it *adds* the cluster-aware tiebreaks `-score` and `-|cluster|` (which LPM cannot compute—it has no pending tree).

Against the thrashing failure mode (§2), cLPM’s score ordering keeps a larger cluster’s members together *within* their section, amortizing each prefill across multiple hits—making cLPM the dominant cache-hit lever (within 3 pp of the full PEEK stack, §4.1). PEEK additionally supports an optional strengthening, *group-major* (GM), that emits all members of one cluster contiguously (1 prefill + size−1 hits per cluster); cLPM’s score ordering already approximates GM grouping in typical queues, so GM adds only marginal cache hit.

We observed that GM helps cache hit but aggressively defers singletons. To rescue them, PEEK introduces a *fairness lane* and a *dynamic-lane* (DL) controller: cLPM stride-interleaves Lane *A* (Eq. 1) with the fairness lane keyed by (section, arrival, $-h$): a fraction α of admissions come from Lane *A*, $1-\alpha$ from Lane *B*. The oldest waiting request advances at least every $\lceil 1/(1-\alpha) \rceil$ cycles. Default $\alpha=0.7$. DL adjusts α per cycle from queue composition—`singleton_frac` (fraction in no cluster) and `age_pressure` = $\min(1, \text{oldest_singleton_wait}/2s)$ —ramping the fairness share $1-\alpha$ from 0.15 (cluster-dominated) toward 0.60 (singleton-heavy or near-SLO), EMA-smoothed:

$$1-\alpha \leftarrow \text{EMA}_{\beta=0.3}(\text{clamp}(0.15 + 0.5 \cdot \text{singleton_frac} + 0.3 \cdot \text{age_pressure}, 0.10, 0.60)).$$

For eviction, PEEK’s hook (PE) overrides LRU/LFU when the engine must free memory, using the same pending tree: candidate block b with tokens t_b scores

$$\text{evict_score}(b) = \max_{v \in \text{ancestors}(t_b)} \text{pending_count}(v) \cdot \text{depth}(v), \quad (2)$$

depth-weighted demand along b ’s ancestor chain (max captures deep lightly-shared prefixes sitting above heavily-shared subtrees); lowest score evicts first. Of four scoring modes (*plain*, *cluster*, *recency*, *decay*), experiments use *cluster*. SGLang installs a custom `EvictionStrategy`; vLLM patches `BlockPool.get_new_blocks` to prefer zero-demand victims (capped at $4N$).

In summary, DL’s clamp $[0.10, 0.60]$ keeps cache locality dominant (Lane *A* retains at least 40%) while EMA damps oscillation, rescuing singletons GM defers; PE protects what waiting clusters need by evicting zero-demand blocks first and keeping high-demand blocks last. Together, scheduling and eviction stay aligned through the same pending tree.

3.3 Guards

PEEK’s per-cycle cost (dual-walk + sort) would otherwise apply even when the queue has no sharing. The pending tree exposes a `has_sharing` primitive—an $O(\# \text{root-children})$ scan returning true iff some root-child has `pending_count` ≥ 2 —available for callers to short-circuit cLPM. In addition, on SGLang the stock scheduler’s queue-depth fallback (LPM degenerates to FCFS once the waiting queue exceeds 128 requests) provides a second cheap exit. Empirically these two paths together leave PEEK matching FCFS within $\pm 1.5\%$ on workloads with no exploitable structure (W4, W5; §4.4); the dual-walk + sort cost is amortized to the noise floor in those regimes.

4 Experiments

We evaluate PEEK on a five-workload suite (W1–W5) covering typical production LLM serving scenarios across diverse models, datasets, and single- and multi-GPU hardware (Appendix Tables 4–6, 9). Each workload comprises one or more *cells*—parameter configurations along its primary axis—each run at *moderate* and *heavy* load.⁴ We report means across 3 seeds after ~ 10 –20% warmup; Table 2 lists policy labels. W1 (§4.1) sweeps the full ablation lattice and identifies cLPM+GM+DL+PE as PEEK’s most performant policy; W2 (§4.2), W3 (§4.3), and W4–W5 (§4.4) probe orthogonal axes using cLPM+GM+DL+PE alone, conserving GPU resources.

4.1 W1: Shared-Prompt Chat Across Production KV-Pressure Tiers

W1 tests whether PEEK’s co-design holds up as KV-cache pressure rises. Cells A/B/C/D span $2\times$ to $16\times$ KV oversubscription, each mapped to a production class (Appendix Tables 7–8). Figures 2–3 plot cache hit and TTFT; throughput and E2E are in Appendix Figures 8, 9 and full numerical results in Appendix Tables 10–12. We first use cLPM+GM+DL+PE to compare against the stock baselines.

⁴Moderate targets sustained queue length 60–127; heavy targets ≥ 128 , the threshold at which SGLang’s LPM falls back to FCFS. The same targets are used on vLLM for cross-engine comparability.

Policy labels		
Label	Components (scheduler + eviction)	Role
FCFS+LRU	FCFS + LRU	Naïve baseline (no prefix caching)
LPM+LRU	SGLang LPM + LRU	SGLang baseline
FCFS(APC)+LRU	vLLM FCFS + APC + LRU	vLLM baseline
LPM+PE / FCFS(APC)+PE	stock scheduler + PEEK PE	Eviction-only ablation
cLPM	cLPM + LRU	Sort-key-only ablation
cLPM+GM	cLPM + group-major + LRU	Scheduling (sort + admission)
cLPM+GM+PE	cLPM + group-major + PEEK PE	Primary co-design
cLPM+GM+DL+PE	cLPM + group-major + dynamic-lane + PE	+ starvation guard

Table 2: Policy labels used in Section 4.

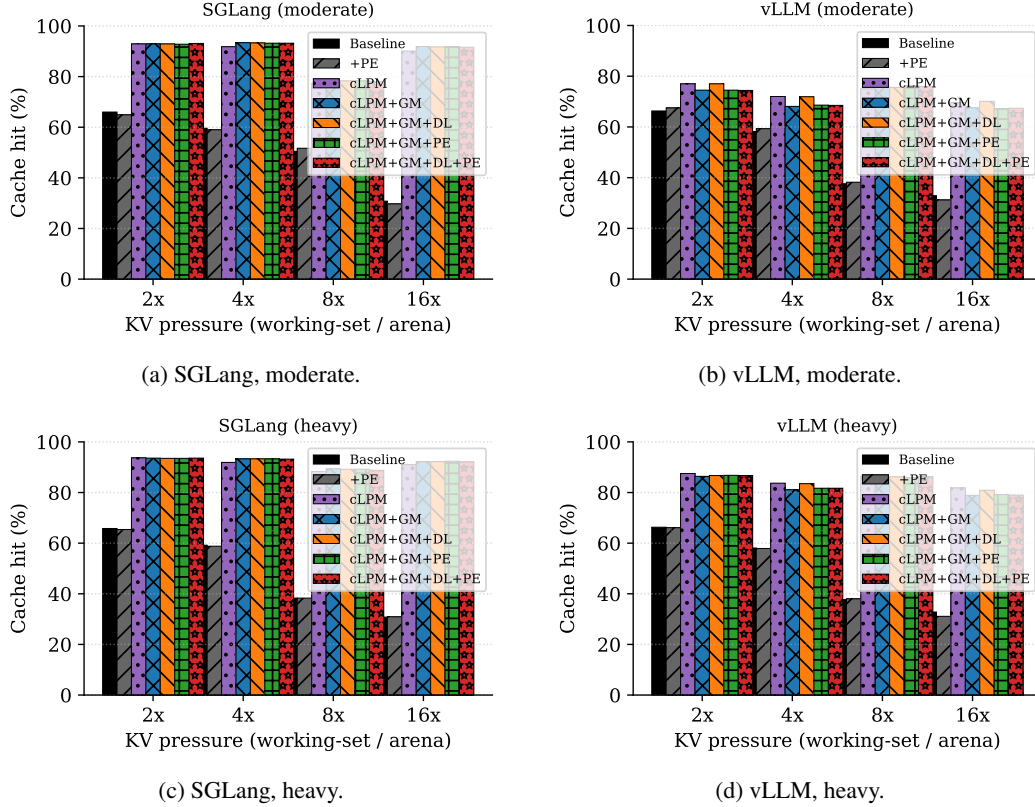


Figure 2: W1 cache hit rate vs. KV pressure.

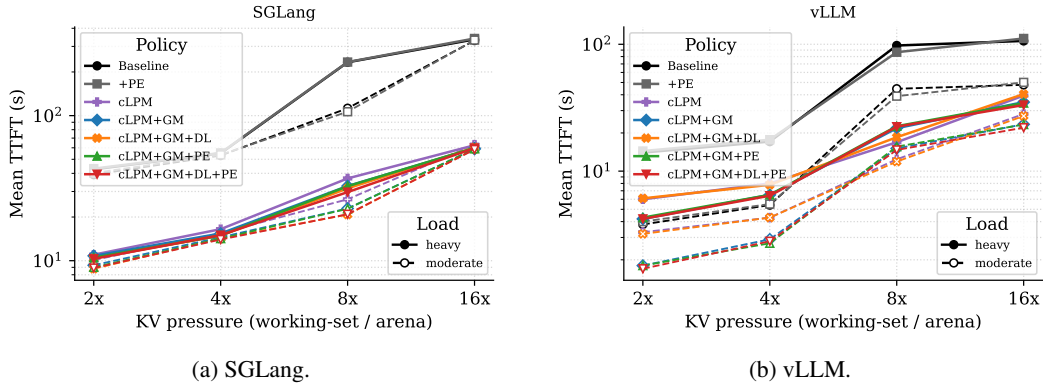


Figure 3: W1 mean TTFT vs. KV pressure (log y-axis). Each panel overlays both load levels: solid lines with filled markers = heavy, dashed lines with hollow markers = moderate.

cLPM+GM+DL+PE lifts cache hit by 28–61 pp (SGLang, heavy) / 20–49 pp (vLLM, heavy; moderate cells start as low as 8 pp)—**1.4–3.0 $\times$ /1.3–2.4 $\times$** —and these gains transfer to TTFT **3.7–7.9 $\times$ /1.9–4.4 $\times$** , E2E **2.9–6.7 $\times$ /1.2–3.6 $\times$** , and throughput **1.5–3.6 $\times$ /1.2–2.5 $\times$** (SGLang/vLLM). Gains scale with KV pressure as baselines thrash (§2) while cLPM+GM+DL+PE stays flat, except at 16 $\times$ heavy load where over-saturation narrows the ratio. Decode-side (Appendix Table 12): TPOT runs **8–26% (4–12 ms/token)** above baseline; ITL tracks TPOT closely (drift +3.6 to +11.5 ms across cells) and moves with throughput rather than per-token cost—both costs are dwarfed by the E2E gains.

cLPM alone contributes most of PEEK’s gain—within **3 pp** of the full PEEK stack on cache hit on both engines—confirming that cluster-aware admission is the dominant lever among PEEK’s mechanisms. The mechanism behind the gain: by replacing stock LPM’s FCFS-on-ties with pending-tree-aware tiebreaking, cLPM admits cluster siblings adjacent to their pioneer, so each shared-prefix prefill is amortized across many cache hits before any unrelated arrival can evict the prefix.

cLPM+GM lifts SGLang cache hit **–0.1 to +1.4 pp** over cLPM but trails the best on TTFT/E2E by up to **10%/8%**; on vLLM it *reduces* cache hit by **0.4–3.0 pp** and trails on TTFT/E2E by up to **29%/22%** at cell C. **GM, aggressive clustering, buys little cache hit but inflates TTFT/E2E**: cLPM already heads the queue with cluster pioneers, and GM locks in the entire cluster as a contiguous batch—amortizing the shared prefill across siblings while deferring singletons.

DL shortens TTFT/E2E by **2.4–15.1%** on SGLang A/C and vLLM C, **recovering the singleton-starvation cost that GM imposes**; elsewhere it trades a small TTFT regression for queue-aging fairness. PE adds **0–3%** on SGLang and is flat on vLLM ($\pm 5\%$); on a stock scheduler PE is flat (± 1 pp, $\pm 2\%$)—**eviction cannot create temporal locality the scheduler did not first establish**.

Takeaway 1. For high-sharing workloads, all cLPM-based PEEK policies substantially outperform stock baselines on both engines on cache hit, TTFT, E2E, and throughput, with the gain scaling as KV pressure rises. The biggest gain comes from cLPM’s cluster-aware scoring atop stock LPM. GM further pushes cache hit but at the cost of delaying singletons, causing a noticeable loss on TTFT and E2E. DL and PE recover those two ends, ultimately establishing cLPM+GM+DL+PE as the most performant policy on both engines.

4.2 W2: Long-Document RAG under Decode-Dominant KV Pressure

W2 tests PEEK on long-prefix RAG, where documents overflow the KV arena (Appendix Table 6). Figure 4 plots cache hit and TTFT; throughput and E2E are in Appendix Figures 10, 11 and full numerical results in Appendix Tables 14–15.

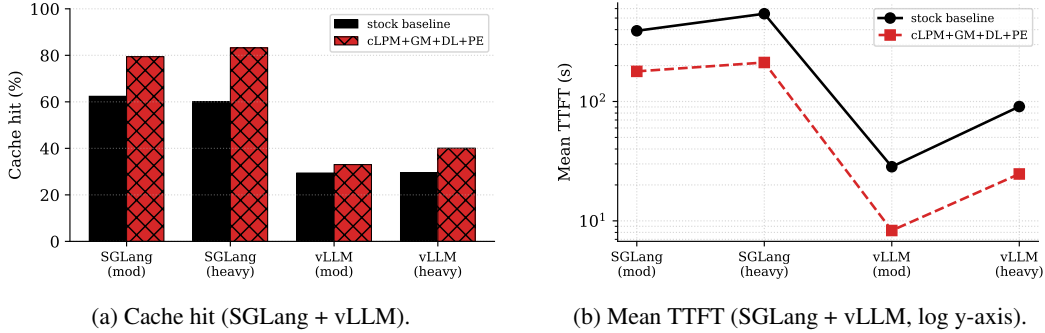


Figure 4: W2 cache hit and mean TTFT, both engines and load levels.

cLPM+GM+DL+PE substantially outperforms the per-engine baseline on long-prefix RAG. Cache hit lifts **+17/+23 pp** (SGLang, mod/heavy) and **+3.6/+10.6 pp** (vLLM)—equivalently **1.27 $\times$ /1.39 $\times$** and **1.12 $\times$ /1.36 $\times$** . These translate into TTFT reductions of **2.2 $\times$ /2.6 $\times$** on SGLang and **3.4 $\times$ /3.7 $\times$** on vLLM, mean E2E reductions of **2.0 $\times$ /2.4 $\times$** and **1.6 $\times$ /2.3 $\times$** , and throughput rising **1.14 $\times$ /1.26 $\times$** on SGLang and **1.00 $\times$ /1.06 $\times$** on vLLM. The large TTFT and E2E reduction (**2 $\times$ –3.7 $\times$**) on both engines reflects the same pattern as W1: under thrashing, low cache hit both inflates per-request prefill and blocks the queue, so cLPM+GM+DL+PE accelerates admitted requests *and* clears the queue faster (§2). Decode-side, mean TPOT moves within $\pm 5\%$ on SGLang and $\pm 9\%$ on vLLM; mean ITL agrees with TPOT to within **1 ms** in every cell—the gain comes from prefill recovery rather than decode-side optimization.

Takeaway 2. As in W1, cLPM+GM+DL+PE wins large on both engines, confirming **PEEK’s advantage transfers to long-decode RAG scenarios.**

4.3 W3: Multi-Cell Shared-Prompt Workloads (Multi-GPU, DP=1 and DP=2)

W3 scales W1 and W2 to Llama-3.1-70B-Instruct (TP=2) at two data-parallel scales—DP=1 (single replica, 2×H100) and DP=2 (two replicas, 4×H100)—across two regimes: *cell B* mirrors W2 (RAG-like, decode-bound) and *cell C* mirrors W1 (chat-like, admission-bound); see Figure 5, Appendix Figures 12–13 (§A.2.8), Appendix Tables 16–17 (§A.2.7), and Appendix Table 6.

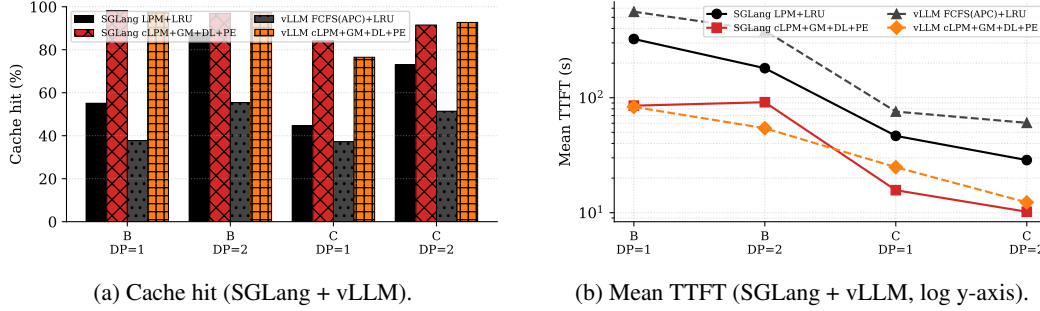


Figure 5: W3 cache hit and mean TTFT across two cells (B, C) and two topologies (DP=1 / 2×H100, DP=2 / 4×H100), heavy load.

cLPM+GM+DL+PE wins large at *both* topologies. At DP=1, SGLang cache hit lifts +43.1/+39.6 pp on cell B/C, TTFT 3.80×/2.97×, E2E 3.62×/2.65×, throughput 2.66×/2.08×; vLLM cache hit lifts +59.4/+39.3 pp, TTFT 6.73×/3.04×, E2E 5.54×/2.58×, throughput 4.45×/2.07×. Doubling to DP=2 mostly helps the stock baseline: SGLang baseline cache hit jumps +32.7/+28.3 pp from DP=1 to DP=2 (vLLM +17.7/+14.2 pp), while PEEK is already at ceiling on cell B (96.9–98.2% SGLang, 97.1–97.3% vLLM) and reaches 84–92% on cell C. The relative gap thus narrows on SGLang to +9.1/+18.6 pp (still up to 2.81× TTFT, 1.62–1.76× throughput) but stays nearly flat on vLLM (+41.9/+41.3 pp; up to 7.08× TTFT, 4.50×/2.83× throughput)—APC’s block-hash misalignment doesn’t shrink with KV space. Notably, PEEK at DP=1 matches or beats the stock baseline at DP=2 on cell B for both engines (SGLang: 98.2 vs 87.8% cache, 85 vs 181 s TTFT, 1.24 vs 1.04 req/s; vLLM: 97.1 vs 55.4%, 83 vs 384 s, 1.07 vs 0.48 req/s)—PEEK delivers more on half the GPUs than the per-engine baseline does on four. Decode-side, TPOT moves within ±7% on SGLang (cell C/DP=1 the worst at −6.9%); on vLLM TPOT rises with throughput on PEEK runs (larger decode batch inflates per-token time in absolute terms, but prefill dominates and E2E still drops sharply).

Takeaway 3. cLPM+GM+DL+PE wins large at both DP=1 and DP=2 on Llama-3.1-70B. **PEEK’s gain scales to larger models and multi-GPU topologies.**

4.4 No-Regress on Prefix-Coherent and Singleton-Dominated Traffic (W4, W5)

W4 and W5 establish PEEK’s no-regress claim from two angles. **W4** runs an already-prefix-coherent scenario (*agentic_only*: multi-turn bursts; *agentic_shared*: 1402-token shared system prompt), where session-burst adjacency leaves little room to improve. **W5** runs heterogeneous singleton chat in *cell_C_long* (512–4096 tokens, weak residual overlap) and *cell_C_short* (32–1024 tokens, no structural sharing). Full results in Appendix §A.2.9–A.2.10 (W4) and §A.2.12–A.2.11 (W5).

W4 (already prefix-coherent). cLPM+GM+DL+PE matches or marginally beats the strongest stock baseline (LPM+LRU on SGLang, FCFS(APC)+LRU on vLLM) on every metric in every cell: SGLang cache hit +0.3–1.0 pp, TTFT within ±5%, E2E −0.8–2.5%, throughput +1.1%; vLLM cache hit +0.0–1.7 pp, TTFT within ±7%, E2E and throughput within ±2%. TPOT moves −0.3–3.5% (SGLang) / ±2% (vLLM). Marginal wins land where arrival order mismatches prefix order: SGLang on *agentic_shared*, vLLM on *agentic_only* (APC hash collisions).

W5 (singleton-dominated). On *cell_C_long*, cluster-aware sort detects weak overlap LPM’s exact match misses: SGLang cache hit +9.5 pp (0.7%→10.2%), TTFT −11%, E2E −8%, throughput +5%; vLLM TTFT 2.6× reduction, E2E −20%, throughput +21% (APC reuse 97.5%→98.8%). On *cell_C_short*, the *has_sharing* guard (§3.3) short-circuits PEEK to FCFS: every metric within ±2% except SGLang TTFT +1.7% and vLLM TTFT −10% relative (0.04 s absolute).

Takeaway 4. PEEK is no-regress on W4 and W5: the `has_sharing` guard short-circuits to FCFS when no structure exists, while cluster-aware admission captures residual overlap when it does. Net: within $\pm 2\%$ (W4) and $\pm 1\%$ (W5) of the strongest baseline, with modest wins on weak structure—complementing W1–W3, where PEEK *recovers* locality the baseline destroyed.

5 Related Work

Prefix-aware request scheduling. *Online:* Preble [Srivatsa et al., 2025] co-optimizes KV reuse with distributed load balancing across replicas. Fu et al. [2024] reorder by predicted output length to reduce head-of-line blocking. DLPM [Cao et al., 2025] adds a delay-bound fairness contract on top of LPM. Dexter et al. [2025] prove the prefix-aware scheduling problem NP-hard and propose a k -LPM approximation. HotPrefix [Li et al., 2025] drives admission and CPU/GPU promotion from a moving average of past hits (*cache hotness*). SOLA [Hong et al., 2025] arbitrates prefix grouping against per-request SLOs. Autellix [Luo et al., 2025] reorders at the program level for agent workloads, capturing inter-call dependencies. *Offline / batch:* BatchLLM [Zheng et al., 2024b] groups requests by global prefix sharing for high-throughput batched inference. BlendServe [Zhao et al., 2026] reorders an offline workload to jointly maximize prefix/KV reuse and balance compute-vs. memory-bound phases, exploiting full visibility into the request set before submission. PEEK differs by forming groups from the *pending queue itself* via an incremental prefix trie—robust to cold, stale, or contended cache—and uses the same trie for eviction so the two co-optimize one signal; it is intra-replica and composes beneath cross-replica routers, fairness contracts, or SLO arbiters. PEEK-offline (Appendix A.3) sits in the BatchLLM/BlendServe regime; this paper targets the online case with no full-workload visibility.

KV cache reuse across prompts. A separate axis supplies the *capability* to share KV across prompts: PromptCache [Gim et al., 2024] (module-level KV via positional re-encoding), RAGCache [Jin et al., 2024] (hierarchical RAG cache), ChunkAttention [Ye et al., 2024] (attention over a prefix tree), Hydragen [Juravsky et al., 2024] (fused shared-prefix kernel), CachedAttention [Gao et al., 2024] (session-scoped state), CacheBlend [Yao et al., 2024] (block reuse across retrieved chunks), CacheGen [Liu et al., 2024] (compressed KV streaming), LMCache [LMCache contributors, 2025] (distributed shared KV). These are mechanisms for *how* KV is stored; PEEK is the policy for *which* blocks are cached and in what order requests run, composing atop any of them.

Within-sequence KV compression and eviction. H2O [Zhang et al., 2023] keeps heavy-hitter tokens by attention history; SnapKV [Li et al., 2024] clusters and prunes tokens at prefill; Scissorhands [Liu et al., 2023] drops decayed-attention tokens; InfiniGen [Lee et al., 2024] offloads cold KV with prefetch; KIVI [Yuan et al., 2024] quantizes KV to 2-bit. These act per-request along the sequence dimension; PEEK acts across requests along the block-allocation dimension and is fully orthogonal.

Disaggregated and distributed serving. DistServe [Zhong et al., 2024] and Splitwise [Patel et al., 2024] disaggregate prefill from decode; Mooncake [Qin et al., 2025] extends this to a distributed KV pool; Llumnix [Sun et al., 2024] migrates running requests across instances. These reshape *where* requests run; PEEK reshapes *when* they run within a replica, and the two axes compose.

Cross-replica prefix-affinity routing. Gateways such as llm-d [Red Hat and IBM and Google contributors, 2025] score backends by prefix-affinity, queue depth, and KV utilization to route requests to the replica most likely to hit warm cache. This is cross-replica; PEEK improves intra-replica scheduling and sits beneath such a router.

Future-aware cache management. KVFlow [Pang et al., 2025] predicts future cache needs from workflow-graph structure; Marconi [Pan et al., 2025] forecasts reuse from prior traffic. PEEK shares the goal but derives demand from the live pending queue via direct reference counting—no execution graph, no learned forecast—and is complementary across within-tick (PEEK) and across-tick (KVFlow/Marconi) horizons.

6 Conclusion

PEEK treats the waiting queue as a first-class signal for KV cache management via an incremental radix tree, a dual-walk yielding all-LPM in $O(C \times D)$, cluster-aware admission/eviction, and an adaptive fairness lane. Across five workloads on SGLang/vLLM up to $4 \times \text{H100}$, PEEK lifts cache hit $3.0 \times / 2.6 \times$, TTFT $7.9 \times / 7.1 \times$, E2E $6.7 \times / 5.5 \times$, throughput $3.6 \times / 4.5 \times$, matches baselines

on no-sharing traffic, and composes beneath cross-replica routers and SLO arbiters atop any prefix-reuse mechanism. *Limitations*: the N/C amortization shrinks toward 1 in singleton-dominated traffic (the `has_sharing` guard handles this regime); the cLPM sort still runs in Python; evaluation is on H100 only, leaving smaller-memory accelerators and multi-modal prefixes for future work. Code will be released.

References

- Shiyi Cao, Yichuan Jiang, Zhiqiang Xie, and Lianmin Zheng. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.
- Gregory Dexter, Sen Ye, Aditya Shanbhag, Ion Stoica, and Hao Zhang. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Yichao Fu, Siqi Xue, Yongji Dai, Silei Qiu, Xinyi Sui, Ion Stoica, and Hao Zhang. Efficient LLM scheduling by learning to rank. *Advances in Neural Information Processing Systems*, 37, 2024.
- Bin Gao, Zhuomin Yin, Yifeng Deng, Jianhui He, Zhengda Liu, Zhenzhong Sun, Peng Li, and Yu Cheng. Cost-efficient large language model serving for multi-turn conversations with Cache-dAttention. 2024.
- In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 2024.
- Yuxin Hong et al. SOLA: Optimizing SLO attainment with state-aware scheduling for LLM serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Lam, Huanchen Mao, and Meng Li. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.
- Jordan Juravsky, James Bradbury, Jascha Sohl-Dickstein, and Behnam Neyshabur. Hydragen: High-throughput LLM inference with shared prefixes. *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- Wonbeom Lee, Jungi Lee, Junghwan Seo, and Hwangyu Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. 2024.
- Hao Li, Yi Gu, Jun Huan, et al. HotPrefix: Hotness-aware KV cache scheduling for LLM serving. *Proceedings of the ACM on Management of Data (PACMOD)*, 2025.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37, 2024.
- Yuhan Liu, Hanchen Li, Kuntai Du, Jiayi Yao, Yihua Cheng, Yuyang Huang, Shan Lu, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2023.
- LMCache contributors. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.
- Michael Luo et al. Autellix: An efficient serving engine for LLM agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- Rui Pan et al. Marconi: Prefix caching for the era of hybrid LLMs. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- Shizhe Pang, Yingchao Zhu, Mengwei Huang, Yubin Zeng, Jiaqi Xu, Siyuan Wen, Dongming Zhang, Zhenghua Cai, and Quan Chen. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. *Advances in Neural Information Processing Systems*, 38, 2025.

- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *51st International Symposium on Computer Architecture (ISCA)*, 2024.
- Ruoyi Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Yang, Weimin Zheng, and Zhengyu Zha. Mooncake: A KVCache-centric disaggregated architecture for LLM serving. *USENIX Conference on File and Storage Technologies (FAST)*, 2025.
- Red Hat and IBM and Google contributors. llm-d: Kubernetes-native distributed inference with KV-cache-aware routing, 2025. Inference Gateway / Endpoint Picker; <https://llm-d.ai>.
- Vikranth Srivatsa, Zijian Ding, Zhenyu Wang, Yicheng Hu, Cedric Xia He, Yuan Tian, and Minlan Wang. Preble: Efficient distributed prompt scheduling for LLM serving. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Biao Sun, Ziming Huang, Hanyu Chen, Jiaming Shi, Wenli Chen, Xingjian Liao, Zhenming He, and Yongji Xiao. Llumnix: Dynamic scheduling for large language model serving. 2024.
- Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*, 2024.
- Lu Ye, Ze Dai, Hao Cao, and Erwei Chen. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. *arXiv preprint arXiv:2402.15220*, 2024.
- Zirui Yuan, Jingbo Shang, Yang Song, Yifei Wu, Peng Qi, and Albert Gu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2023.
- Yilong Zhao, Shuo Yang, Kan Zhu, Lianmin Zheng, Baris Kasikci, Yifan Qiao, Yang Zhou, Jiarong Xing, and Ion Stoica. BlendServe: Optimizing offline inference with resource-aware batching. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2026.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Yineng Liang, Siyuan Zhuang, et al. SGLang: Efficient execution of structured language model programs. In *Proceedings of the 2024 Conference on Neural Information Processing Systems (NeurIPS)*, 2024a.
- Zhen Zheng, Xin Han, Zhiyu Yang, Shulin Wu, Fan Zhang, Jianbo Zhu, Mofan Yang, Daoguang Zha, Jian Liu, Gaofeng Yang, and Lixin Zhang. BatchLLM: Optimizing large batched LLM inference with global prefix sharing and throughput-oriented token batching. *arXiv preprint arXiv:2412.03594*, 2024b.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

A.1 PEEK: Predictive Queue-Informed KV Cache Management Mechanisms

A.1.1 PEEK vs. SGLang LPM: Mechanism Comparison

PEEK keeps stock LPM’s longest-prefix-match objective and reuses its pioneer/sibling threshold ($\theta=32$ tokens), but otherwise replaces every other component with a pending-tree-driven counterpart. Table 3 lays out the differences axis by axis; each row maps to a section of the main text or this appendix.

A.1.2 Implementation Details

Native core. The pending radix tree, dual-walk, and per-rid cluster/score lookups are implemented in ~ 1.6 kLOC of Rust (`src/tree.rs`, `src/pending.rs`) and exposed to Python via PyO3 (`peek._core`). Nodes live in a contiguous arena indexed by `NodeId`, with `FxHashMap` children keyed

Axis	SGLang LPM (stock)	PEEK (cLPM + PE + DL)
Prefix signal source	Engine-side radix cache only — <i>reactive</i> : ranks by what is currently cached	Engine cache <i>plus</i> a pending-queue radix tree — <i>predictive</i> : ranks by what waiting requests will share (§3.1)
Scheduler sort key	$(-h(r), \text{is_depr}(r), \text{arrival}(r))$	$(\text{section}(r), -h(r), -\text{score}(r), - \text{cluster}(r) , \text{arrival}(r))$ (§3.2)
Cold-tie behavior ($h=0$)	Falls back to arrival (FCFS) — interleaves clusters, induces thrashing (§2)	Largest-cluster pioneer first; siblings sort directly behind, so one prefill amortizes across the whole cluster
In-batch deprioritization	Auxiliary radix tree rebuilt from scratch each scheduling call (throw-away)	Read off the persistent pending tree at $O(1)$ per request — no rebuild
Per-cycle LPM cost	$O(N \times D)$ per-request cache lookups	$O(C \times D)$ via tree-vs-tree dual-walk on SGLang ($C = \#$ distinct shared prefixes, $C \ll N$ in high-sharing queues); reuses engine probe on vLLM (§3.1)
Cluster signals exposed	None	Cluster identity, cluster size, depth-weighted ancestor demand score
Eviction policy	LRU/LFU on cache state only — decoupled from scheduling	Queue-aware hook scoring blocks by $\max_{v \in \text{ancestors}} \text{pending_count}(v) \cdot \text{depth}(v)$: zero-demand blocks first, blocks the largest pending clusters depend on protected last (§3.2)
Starvation handling	None beyond the arrival tiebreak	Stride-interleaved fairness lane ($\text{section}, \text{arrival}, -h$); a dynamic-lane controller widens the fairness share with singleton fraction and oldest-singleton wait, EMA-smoothed (§3.2)
No-sharing fallback	Always pays the LPM/aux-tree cost	<code>has_sharing</code> guard short-circuits to FCFS at zero overhead when the queue contains no shared prefixes (§3.3)
Scheduling/eviction coupling	Decoupled; eviction cannot anticipate scheduler choices	Both consume one shared signal (the pending tree) — co-designed without explicit coordination

Table 3: PEEK vs. stock SGLang LPM, axis by axis. PEEK preserves LPM’s longest-prefix-match objective and pioneer/sibling threshold but replaces FCFS tiebreaks with cluster-aware ordering, eliminates the per-cycle aux-tree rebuild via the persistent pending tree, drives queue-aware eviction off the same tree, and bounds singleton starvation with an adaptive fairness lane.

by the first edge token, FxHashSet terminators, and per-node `pending_count`; insert/remove run in $O(D)$ with no allocation in the hot path. The cLPM sort itself runs in Python ($O(N \log N)$ over the Rust-computed signals); pushing the tree, dual-walk, and per-rid score/cluster computations into Rust eliminates the per-compare Python callback overhead that dominates stock SGLang LPM at large N .

SGLang integration (~814 LOC of Python). `peek.engines.sglang.patch_hook` monkey-patches two surfaces at import time. `RadixCache.__init__` is patched to install `PeekDemandStrategy` (§3.2) in place of the default heap-based eviction. `Scheduler.process_input_requests` is patched to diff-sync the pending tree against `self.waiting_queue` on every scheduling iteration—a mutation-path-agnostic sync that tolerates any downstream queue manipulation (append, pop, list-comprehension reassignment) without further hooks. The same hook overrides `SchedulePolicy.calc_priority` (the entry point that internally calls `_sort_by_longest_prefix` in stock SGLang) to invoke PEEK’s cLPM sort.

vLLM integration (~800 LOC of Python). `peek.engines.vllm.patch_hook` hooks three v1 scheduler entry points: `Scheduler.schedule` (sync + reorder `self.waiting`), `Scheduler.add_request` (sync-on-arrival fast path), and `Scheduler.finish_requests` (drop

completed rids). Because vLLM’s prefix cache is hash-keyed rather than tree-structured, PEEK additionally maintains an inverted index `block_hash → demand` alongside the pending tree, and patches `BlockPool.get_new_blocks` to scan the free-block queue (capped at $4N$ entries) and prefer zero-demand victims over the default LRU pick. Recency/decay eviction modes also patch `BlockPool.touch` and `cache_full_blocks` to fabricate per-block timestamps that vLLM does not track natively. vLLM v1 spawns its `EngineCore` via `multiprocessing.spawn`, so monkey-patches in the parent do not inherit; PEEK ships a `sitecustomize.py` shim that re-imports the hook in every child interpreter.

Activation surface. A single environment flag selects scope: `PEEK_ONLINE_SCHEDULER` for cluster-aware scheduling, `PEEK_ONLINE_EVICTION` for queue-aware eviction, `PEEK_ONLINE_CLPM` for the multi-lane scheduler, `PEEK_ONLINE_EVICTION_MODE` $\in \{\text{plain, cluster, recency, decay}\}$ for the eviction variant, and `PEEK_ONLINE_CLPM_BIGLANE_SHARE` for the cache-locality lane share (default 0.7). With no flags set, both shims are no-ops and the engine runs unmodified. Because integration is by monkey-patching, no fork or PR against either upstream engine is required, and PEEK tracks SGLang’s main branch and vLLM 0.19.1 with byte-identical baselines.

A.1.3 Pending Radix Tree: Build and Clustering

The pending radix tree (§3.1) is a textbook compressed trie with two augmentations: per-node `pending_count` and per-leaf terminators. `INSERT` descends from the root following matching edges, splitting any edge on token divergence (creating a new internal node) and adding a new leaf for the request, then bumps `pending_count` along the root-to-leaf path— $O(D)$, where D is the prompt’s tree depth. `REMOVE` reverses: walk to the request’s terminator, remove the rid, decrement `pending_count` along the path, and prune empty branches—also $O(D)$. `CLUSTER` walks the request’s prompt down the tree and returns the deepest node v with `pending_count(v) ≥ 2`; cluster size is `pending_count(v)`. The tree therefore costs $O(N \cdot D)$ per cycle in total, dominated in practice by the scheduler’s $O(N \log N)$ sort (§3.2).

A.1.4 Dual-Walk for Longest Prefix Match

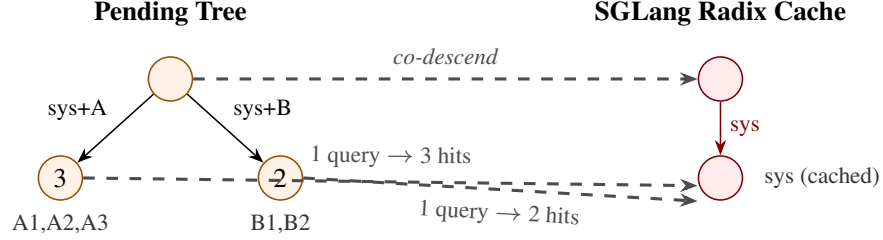
Figure 6 contrasts the two engines. On SGLang’s radix cache, PEEK runs the recursive dual-walk: tree-vs-tree co-descent collapses N per-request lookups into C per-cluster lookups (Algorithm 1). The recursion descends a pending edge only when the edge matches in full—a partial match terminates the branch, since deeper nodes cannot extend the cached prefix. On vLLM’s flat block-hash cache, no structural co-descent is possible: PEEK calls the engine’s per-request `find_longest_cache_hit` once per waiting request to obtain $h(r)$, then walks the pending tree separately to read cluster size and ancestor score—the LPM lookup is not amortized, but the cluster signal stock vLLM’s FCFS lacks is still surfaced.

Cost. On SGLang, each pending edge incurs one cache co-traversal step; total is $O(C \times D)$ across C pending tree edges of average depth D , vs. $O(N \times D)$ for stock LPM—an amortization factor of N/C (large when sharing is high). On vLLM, the LPM lookup itself is not amortized: $O(N \times b)$ for N `find_longest_cache_hit` calls (b = block-hash chain length), matching stock APC’s per-request cost; the additional $O(N)$ pending-tree walk for cluster signals is dwarfed by the LPM probes. Algorithm 1 (§3.1) details the SGLang procedure.

A.1.5 cLPM Scheduling Algorithm

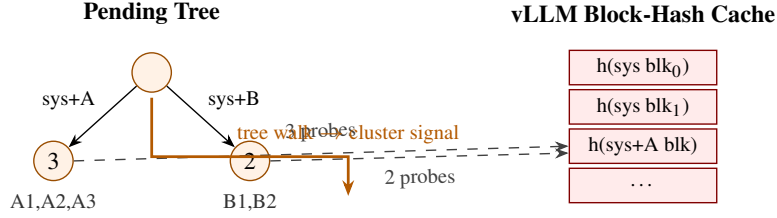
Algorithm 2 details the per-cycle scheduling routine described in §3.2: dual-walk (or per-request LPM on vLLM) for $h(r)$, pending-tree lookups for cluster signals, lexicographic sort of two independent lane keys (cache-locality lane keyed by Eq. 1; fairness lane keyed by arrival), and stride-interleave between them.

Cost. Dual-walk: $O(C \times D)$ on SGLang. On vLLM, the cluster signal is read directly from the pending tree via `all_cluster_info` ($O(N)$) and per-request LPM is satisfied by one $O(b)$ probe each into the engine’s block-hash index (b = block-hash chain length), totaling $O(N \times b)$ for the LPM lookup; the structural pending-tree walk used by SGLang is unnecessary because vLLM exposes a direct flat-cache lookup. Per-request `score/cluster` reads are $O(1)$ (computed once via batched



Stock LPM: 5 queries ($O(N \times D)$). PEEK: 2 queries ($O(C \times D)$).

(a) Dual-walk on SGLang: tree-vs-tree co-descent amortizes lookups across cluster siblings.



Stock vLLM: 5 hash probes, no cluster signal. PEEK: 5 probes + tree walk surfaces clusters for cLPM.

(b) Dual-walk on vLLM: per-request hash probes are unavoidable, but the pending-tree walk surfaces the cluster signal that vLLM's flat hash cache cannot.

Figure 6: Dual-walk on (a) SGLang and (b) vLLM. SGLang's radix cache permits tree-vs-tree co-descent, collapsing N per-request LPM lookups into C per-cluster lookups. vLLM's flat block-hash cache forces one probe per request, but the pending-tree walk still extracts cluster information for the scheduler downstream.

Rust calls). Sort: $O(N \log N)$. Stride interleave: $O(N)$. Total per cycle: $O(C \times D + N \log N)$ on SGLang; $O(N \times b + N \log N)$ on vLLM.

Guard. The pending tree exposes `has_sharing()` (an $O(\# \text{root-children})$ check); when integration is configured to skip cLPM on no-sharing queues, Lines 1–16 are bypassed and the engine's stock order is returned. Empirically PEEK matches FCFS within $\pm 1.5\%$ on no-sharing workloads (§3.3, §4.4), driven by both this guard and SGLang's stock LPM-fallback at queue depth > 128 .

A.1.6 Eviction Motivation Example

Even with good scheduling, uninformed LRU eviction can undo prefix sharing. Consider three prefix groups A, B, C with a cache holding two prefixes. Wave 1 processes A then B, leaving both cached (B most recently used). Wave 2 needs A and C. When C arrives and the cache is full, LRU evicts A—the least recently accessed—but A is exactly what wave 2 needs next, forcing full recomputation. A queue-aware policy inspects pending requests, sees A is needed and B is not, and evicts B instead—preserving A for an immediate hit.

Algorithm 2 cLPM Per-Cycle Scheduling

Require: Waiting queue $\mathcal{R}$, pending tree T , engine cache $\mathcal{C}$, cache-lane share α , warm threshold τ , deprio threshold θ

Ensure: Admission order over $\mathcal{R}$

```
1:  $H \leftarrow \text{dual\_walk}(T, \mathcal{C})$   $\{H[r] = h(r) \text{ for each } r \text{ (Alg. 1)}\}$ 
2:  $S \leftarrow \text{compute\_req\_scores}(T)$ ;  $K \leftarrow \text{all\_cluster\_info}(T)$  {batched Rust calls}
3:  $\text{seen} \leftarrow \emptyset$ 
4: for each  $r \in \mathcal{R}$  in arrival order do
5:   if  $H[r] > \tau$  then
6:      $\text{section}(r) \leftarrow 0$  {warm}
7:   else
8:      $p \leftarrow \text{prompt}(r)[\theta]$ 
9:     if  $p \in \text{seen}$  then
10:       $\text{section}(r) \leftarrow 2$  {cold sibling (deprio tail)}
11:    else
12:       $\text{section}(r) \leftarrow 1$ ;  $\text{seen} \leftarrow \text{seen} \cup \{p\}$  {cold pioneer}
13:    end if
14:  end if
15:   $\text{score}(r) \leftarrow S[r]$ ;  $|\text{cluster}(r)| \leftarrow K[r].\text{size}$ 
16: end for
17:  $\mathcal{R}_A \leftarrow \text{sort}(\mathcal{R}, \text{key} = \text{Eq. 1})$  {cache-locality lane}
18:  $\mathcal{R}_B \leftarrow \text{sort}(\mathcal{R}, \text{key} = (\text{section}, \text{arrival}, -H))$  {fairness lane}
19: if  $\text{dynamic\_lane}$  then
20:    $\alpha \leftarrow 1 - \text{EMA}_{\beta=0.3}(\text{clamp}(0.15 + 0.5 \cdot \text{singleton\_frac} + 0.3 \cdot \text{age\_pressure}, 0.1, 0.6))$ 
21: end if
22: return  $\text{stride\_interleave}(\mathcal{R}_A, \mathcal{R}_B, \alpha)$  {cache-locality lane gets share  $\alpha$  (default 0.7)}
```



(a) LRU evicts A (least recent), but A is needed next. (b) Queue-aware: evicts B (not needed), preserves A.

Figure 7: LRU vs. queue-aware eviction. After wave 1 caches A and B, wave 2 needs A and C. LRU (a) evicts A—the least recently used—forcing recomputation. Queue-aware eviction (b) inspects pending requests, preserves A, and evicts the unreferenced B.

A.2 PEEK Experiments

A.2.1 Experimental Setup Tables

A.2.2 Workload-to-Production-Scenario Mapping

A.2.3 W1 Full-Lattice Heavy-Load Tables

The three tables below report W1’s full 7-policy lattice on heavy load: throughput, cache hit, mean TTFT, and mean E2E on SGLang (Table 10) and vLLM (Table 11); decode-side mean TPOT and ITL on both engines (Table 12). The body’s W1 narrative (§4.1) summarizes these via the figures and ablation paragraphs.

A.2.4 W1 Throughput and E2E Figures

These two figures complement the TTFT figure in the body (Figure 3) and the full-lattice tables in Appendix A.2.3.

Table 4: Hardware, model, and dataset assignment per workload.

W	GPUs	Model (bf16)	Dataset
W1	1×NVIDIA H100 80 GB	Qwen2.5-32B-Instruct	LooGLE long-document corpus: 100 group prompts (~1024 tokens each), Zipf- α =1.0 sampling
W2	1×NVIDIA H100 80 GB	Qwen2.5-32B-Instruct	RepoBench-Python v1.1 (<i>cross_file_first</i> split): 40 repository contexts truncated to ~8192-token shared prefixes, Zipf- α =1.0 sampling
W3	2–4×NVIDIA H100 80 GB (TP=2; DP=1 / DP=2)	Llama-3.1-70B-Instruct	Multi-GPU scaling of W1 and W2: cell B mirrors W2 (long-prefix RAG), cell C mirrors W1 (shared-prompt chat)
W4	1×NVIDIA H100 80 GB	Mistral-Small-24B-Instruct-2501 (matches Mooncake trace)	Mooncake <i>conversation_trace</i> (FAST’25) with within-session prefix accumulation; <i>agentic_only</i> and <i>agentic_shared</i> scenarios
W5	1×NVIDIA H100 80 GB	Gemma-2-27B-it	LMSYS arena chat (heterogeneous singletons): <i>cell_C_long</i> (512–4096 tokens) and <i>cell_C_short</i> (32–1024 tokens)

Table 5: Shared experiment setup across W1–W5.

Component	Configuration
Engines	SGLang 0.5.9, vLLM 0.19.1
Framework	PyTorch 2.9.1, Python 3.12.13, CUDA graphs <i>on</i>
Memory budget	SGLang <code>mem_fraction_static</code> =0.88; vLLM <code>gpu_memory_utilization</code> =0.9
Seeds	42, 142, 242 (3 replicates per cell; reported numbers are means across seeds; $\pm$ values are seed standard deviation)
Warmup	First ~10–20% of requests per cell (50–400 requests depending on cell size) are excluded from metric aggregation
Load levels	Two operating points per cell, calibrated by sustained queue depth: <i>moderate</i> maintains a queue length of ~60–100 requests; <i>heavy</i> sustains ~150–200
Policy lattice (SGLang)	B0 (FCFS+LRU), B1 (LPM+LRU); P_cLpm (cLPM only), P_full (cLPM + group-major admission + queue-aware eviction)
Policy lattice (vLLM)	B2 (FCFS+APC+LRU, strongest stock baseline); P2 (eviction only), P3 (cLPM only), P4 (cLPM + group-major), P5 (P4 + dynamic-lane fairness), P6 (P4 + queue-aware eviction, primary co-design), P7 (P5 + queue-aware eviction)

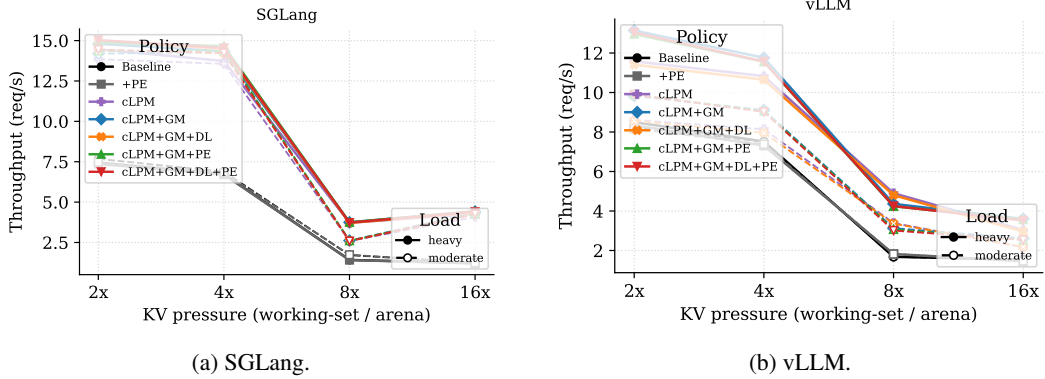


Figure 8: W1 throughput vs. KV pressure. Each panel overlays both load levels: solid lines with filled markers = heavy, dashed lines with hollow markers = moderate.

Table 6: Experiments and per-workload configurations. “Probe axis” names the design facet each workload stresses relative to the W1 driving scenario.

W	Probe axis	Cells	Configuration
W1	Oversubscription (driving)	A: 2 $\times$, B: 4 $\times$, C: 8 $\times$ (primary), D: 16 $\times$ KV pressure; moderate / heavy	Zipf- α =1.0 prefix groups, Poisson arrivals, fixed decode=128. Full ablation lattice (B2, P2–P7).
W2	Long, variable decode (decode-dominant)	cell B (4 $\times$), moderate / heavy	40 documents $\times$ 8192-token prefixes; decode mix 10:128, 25:512, 30:1024, 25:2048, 10:4096 (mean $\approx$ 1460); KV footprint $\approx$ 7 $\times$ arena, eviction always firing.
W3	Multi-GPU scaling (Llama-3.1-70B, TP=2)	cell B (mirrors W2, RAG-like) and cell C (mirrors W1, chat-like) at DP=1 (2 $\times$ H100) and DP=2 (4 $\times$ H100); heavy	Same dataset patterns as W1 (cell C) and W2 (cell B), executed at 70B scale across two data-parallel topologies.
W4	Within-session prefix accumulation (Mooncake)	<code>agentic_only</code> , <code>agentic_shared</code> ; moderate / heavy	Mooncake <code>conversation_trace</code> (FAST’25), 4 rounds, 50 ms inter-turn. <code>agentic_shared</code> adds a 1402-token shared system prompt (Cursor / Copilot / Claude Code pattern).
W5	Singleton-dominated chat (no-regress)	<code>cell_C_long</code> (512–4096 tokens, weak residual overlap), <code>cell_C_short</code> (32–1024 tokens, no structural sharing); heavy	LMSYS arena chat singletons; lognormal length distribution; decode 64–256; Poisson arrivals; N =1500.

Table 7: W1 full configuration. Top: shared across all cells. Bottom: per-cell parameters and per-engine calibrated arrival rates. KV footprint is the peak working-set size; oversubscription is footprint relative to the $\sim$ 50K-token arena. Rates target moderate = queue p50 in [60,127], heavy $\geq$ 128 (the SGLang LPM-fallback boundary).

Shared across all W1 cells											
Hardware	1 $\times$ NVIDIA H100 80 GB (bf16)										
Model	Qwen2.5-32B-Instruct										
Engines	SGLang 0.5.9 (B1=LPM+LRU baseline); vLLM 0.19.1 (B2=FCFS+APC+LRU baseline)										
Dataset	LooGLE long-document corpus, prefix tokens taken from documents										
Group sampling	Zipf- α =1.0 over G shared-prompt groups										
Arrival process	Poisson (inter-arrival $\sim$ Exp(λ))										
Decode	Fixed 128 tokens per request										
Concurrency cap	vLLM: moderate=100, heavy=210; SGLang: 256										
Seeds	42, 142, 242 (3 replicates per cell; reported numbers are means across seeds)										
Warmup	First $N \cdot 0.2$ requests excluded from metrics										
Policies	B1 (SGLang) / B2 (vLLM); P2 (eviction-only); P3 (cLPM); P4 (cLPM+group-major); P5 (P4+dyn-lane); P6 (P4+queue-aware eviction, primary co-design); P7 (P5+queue-aware eviction)										

Cell	G	prefix		warmup		KV (tok)	over-sub.	SGLang (req/s)		vLLM (req/s)	
		G	(tok)	N	(req)			mod.	heavy	mod.	heavy
A	100	1024	1000	200	102 K	2 $\times$		40	60	20	30
B	200	1024	2000	400	205 K	4 $\times$		35	80	20	30
C	100	4096	1000	200	410 K	8 $\times$ (primary)		4	8	5	10
D	200	4096	2000	400	820 K	16 $\times$		12	24	8	15

A.2.5 W2 Heavy and Moderate Load Tables

A.2.6 W2 Throughput and E2E Figures

A.2.7 W3 Heavy-Load Tables (DP=1 and DP=2)

A.2.8 W3 Throughput and E2E Figures (DP=1 and DP=2)

A.2.9 W4 Heavy-Load Tables

A.2.10 W4 Figures

A.2.11 W5 Heavy-Load Tables

A.2.12 W5 Figures

A.3 PFEK-offline

Table 8: W1 cells mapped to production deployment classes. Cell C is designated primary as the most directly representative of PEEK’s target deployment.

Cell	Production analog	Role / question this cell answers
A (2×)	SMB chatbot, single-tenant SaaS (e.g. Intercom Fin AI) where the KV cache mostly fits	“Does PEEK matter at all when KV mostly fits?” — near-baseline reference
B (4×)	Multi-tenant SaaS / mid-size prompt library (Jasper-style content templates; multi-vertical chat platforms aggregating ~200 small tenants)	“Does PEEK help the typical SaaS deployment in steady state?” — common-case production test
C (8×, pri- mary)	Complex prompts at scale: ChatGPT custom GPTs, Claude projects with tool-using assistants, Cursor / Copilot workspace prompts with file/repo context, enterprise platforms with ~100 tool-using personas	“Does PEEK help ChatGPT-class deployments under realistic peak pressure?” — the workload the design directly targets
D (16×)	Large multi-tenant agent platform under flash / extreme load: launch-day API peak, viral-spike chat platform, multi-vertical agent platform during launch	“Does PEEK hold up under flash / overload, or collapse along with everything else?” — stress test

Table 9: W1–W5 mapping to production deployment classes and serving hardware.

W	Class	Production scenarios	GPUs
W1	Shared-prompt / templated apps	SMB chatbots (Intercom Fin), multi-tenant SaaS prompt libraries (Jasper-style), custom-GPT / Claude-projects at scale, enterprise multi-tenant agent platforms under flash load	1×H100
W2	Long-document RAG (decode-dominant)	Enterprise RAG over document corpora (legal, medical, finance); long-form chat-over-doc; reasoning-adjacent synthesis	1×H100
W3	Multi-GPU 70B serving for shared-prompt + RAG	Production deployments of W1- and W2-class workloads on Llama-3.1-70B-class models scaled across DP×TP topologies (single-replica TP=2 and dual-replica DP=2×TP=2)	2–4×H100 (TP=2; DP=1 / DP=2)
W4	Agentic bursts / programmatic serving	Tool-calling assistants (Cursor, Copilot, Claude Code), RAG pipelines with cite-check, reasoning chains (ToT / SoT), computer-use agents, back-end LLM enrichment pipelines	1×H100
W5	Heterogeneous chat (no-regress)	Default ChatGPT / Claude / Gemini chat, open-source hosted chat (HuggingChat, LMSYS Arena), general-purpose inference APIs without enforced templating	1×H100

Table 10: W1 heavy load on **SGLang**: throughput, cache hit rate, mean TTFT, and mean E2E. Means across 3 seeds. PE = PEEK’s queue-aware eviction (`demand_cluster`); GM = group-major admission; DL = dynamic-lane fairness. cLPM alone is the cluster-LPM scheduler with stock LRU eviction (no group-major, no dynamic-lane).

Cell	Metric	LPM+LRU	LPM+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	tput (req/s)	7.43	7.36	14.46	14.80	14.98	14.89	15.03
	cache hit (%)	65.8	65.4	93.7	93.6	93.5	93.5	93.6
	TTFT (s)	42.6	43.1	11.0	10.8	10.4	10.5	10.3
	E2E (s)	47.8	48.3	16.9	16.4	16.0	16.2	15.9
B (4×)	tput (req/s)	6.73	6.70	13.74	14.34	14.49	14.66	14.55
	cache hit (%)	59.2	58.8	91.9	93.3	93.3	93.3	93.2
	TTFT (s)	55.0	55.3	16.5	15.4	14.9	14.9	15.0
	E2E (s)	60.3	60.8	22.5	21.1	20.8	20.8	20.8
C (8×)	tput (req/s)	1.42	1.41	3.72	3.75	3.71	3.75	3.73
	cache hit (%)	38.3	38.3	88.3	89.4	89.2	89.2	88.8
	TTFT (s)	233.2	234.0	36.9	32.7	31.5	32.6	29.7
	E2E (s)	238.2	239.0	42.6	38.3	37.2	38.3	35.3
D (16×)	tput (req/s)	1.22	1.20	4.28	4.42	4.36	4.44	4.41
	cache hit (%)	30.8	30.9	91.0	92.2	92.2	92.4	92.1
	TTFT (s)	335.5	340.2	62.5	59.6	60.3	59.9	59.7
	E2E (s)	340.6	345.4	69.1	66.1	66.7	66.3	66.2

Table 11: W1 heavy load on **vLLM**: throughput, cache hit rate, mean TTFT, and mean E2E. All vLLM cells run with Automatic Prefix Caching (APC) enabled. Means across 3 seeds. Column abbreviations: PE = PEEK’s queue-aware eviction; GM = group-major admission; DL = dynamic-lane fairness.

Cell	Metric	FCFS(APC)+LRU	FCFS(APC)+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	tput (req/s)	8.50	8.34	11.58	13.14	11.41	12.96	13.04
	cache hit (%)	66.3	66.1	87.5	86.4	86.7	86.8	86.7
	TTFT (s)	14.0	14.4	6.0	4.2	6.1	4.3	4.2
	E2E (s)	19.3	19.9	11.1	10.4	11.2	10.7	10.4
B (4×)	tput (req/s)	7.52	7.31	10.83	11.76	10.65	11.56	11.57
	cache hit (%)	58.1	57.9	83.7	81.1	83.5	81.7	81.7
	TTFT (s)	17.1	17.7	8.0	6.5	7.8	6.5	6.4
	E2E (s)	22.4	23.1	13.2	12.6	13.1	12.8	12.5
C (8×)	tput (req/s)	1.68	1.82	4.90	4.36	4.80	4.24	4.25
	cache hit (%)	37.7	38.0	86.5	86.2	86.2	85.8	86.3
	TTFT (s)	97.8	86.6	16.9	21.8	18.5	22.6	22.4
	E2E (s)	102.6	91.5	22.4	27.2	23.8	28.7	28.5
D (16×)	tput (req/s)	1.54	1.46	3.04	3.60	2.96	3.57	3.52
	cache hit (%)	32.8	31.1	81.8	78.9	80.9	79.2	79.0
	TTFT (s)	106.3	111.1	39.4	35.0	40.5	34.4	33.2
	E2E (s)	111.3	116.3	44.4	40.4	45.6	40.4	39.0

Table 12: W1 heavy load decode-side metrics: mean TPOT and mean ITL on both engines. Means across 3 seeds. TPOT and ITL agree to within 0.1 ms in every cell. All vLLM cells run with APC enabled. Column abbreviations: PE = PEEK’s queue-aware eviction; GM = group-major admission; DL = dynamic-lane fairness.

SGLang								
Cell	Metric	LPM+LRU	LPM+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	TPOT (ms)	45.9	46.7	52.0	49.4	49.6	50.9	50.5
	ITL (ms)	45.8	46.5	51.9	49.3	49.6	50.9	50.4
B (4×)	TPOT (ms)	46.9	47.6	52.4	50.2	51.4	52.1	50.5
	ITL (ms)	46.9	47.7	52.3	50.2	51.3	52.0	50.5
C (8×)	TPOT (ms)	44.4	44.4	49.9	49.4	49.9	50.2	49.1
	ITL (ms)	44.5	44.5	50.0	49.4	49.9	50.2	49.1
D (16×)	TPOT (ms)	44.1	45.0	56.0	55.5	55.4	54.9	55.6
	ITL (ms)	44.2	45.0	56.0	55.5	55.3	55.0	55.7

vLLM								
Cell	Metric	FCFS(APC)+LRU	FCFS(APC)+PE	cLPM	cLPM+GM	cLPM+GM+DL	cLPM+GM+PE	cLPM+GM+DL+PE
A (2×)	TPOT (ms)	43.5	44.3	44.3	50.0	44.8	51.7	50.3
	ITL (ms)	43.5	44.3	44.2	50.0	44.6	51.4	50.2
B (4×)	TPOT (ms)	42.2	43.2	44.8	49.4	45.1	51.1	49.8
	ITL (ms)	42.2	43.2	44.5	49.4	45.0	51.0	49.8
C (8×)	TPOT (ms)	39.5	39.0	42.7	43.1	42.4	49.3	49.3
	ITL (ms)	39.4	39.0	43.0	43.1	42.4	49.3	49.3
D (16×)	TPOT (ms)	40.5	41.8	42.1	43.7	42.5	48.4	46.2
	ITL (ms)	40.5	41.8	42.1	43.7	42.5	48.3	46.7

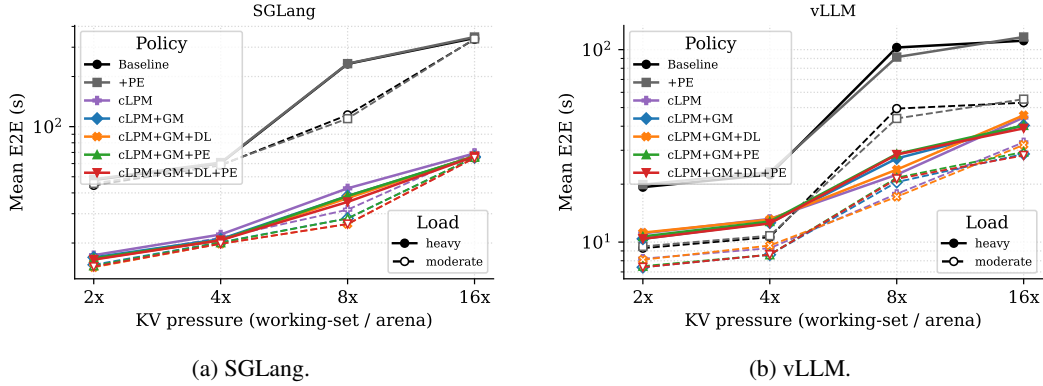


Figure 9: W1 mean E2E latency vs. KV pressure (log y-axis). Conventions as in Figure 3. Same pattern: baselines climb, PEEK stays bounded.

this by replacing the per-cycle pending tree of PEEK-online with a one-shot, full-batch construction, executed entirely as client-side preprocessing through the engine’s public API.

Table 13: W2 full configuration. Top: shared across all W2 runs. Bottom: per-engine, per-load offered arrival rate and request-count; both rates target sustained queue depths matching the paper-wide moderate/heavy definitions and run slightly above r_{sat} (errored counts in single digits). The repository excerpts (Python files within shared repos) provide stable shared prefixes that exceed the model’s KV arena at 8192 tokens, exercising eviction in steady state.

Shared across W2 (cell B, $7 \times$ KV pressure)						
Hardware	1 $\times$ NVIDIA H100 80 GB (bf16)					
Model	Qwen2.5-32B-Instruct					
Engines	SGLang 0.5.9 (B1=LPM+LRU baseline); vLLM 0.19.1 (B2=FCFS+APC+LRU baseline)					
Dataset	RepoBench-Python v1.1 (cross_file_first split): $G=40$ repository contexts, each truncated to a 8192-token shared prefix; per-request continuation question appended after the shared prefix					
Group sampling	Zipf- $\alpha=1.0$ over the $G=40$ prefixes					
Arrival process	Poisson (inter-arrival $\sim \text{Exp}(\lambda)$)					
Decode	Variable, sampled from the mix 10:128, 25:512, 30:1024, 25:2048, 10:4096 (mean ≈ 1460 tokens per request)					
Concurrency cap	256 (both engines)					
Seeds	42, 142, 242 (3 replicates per cell; reported numbers are means across seeds)					
Warmup	First 100 requests excluded from metrics					
Memory budget	SGLang mem_fraction_static=0.88; vLLM gpu_memory_utilization=0.9					
Policies	B1 (SGLang) / B2 (vLLM); cLPM+GM+DL+PE (full PEEK co-design)					

Load	SGLang			vLLM		
	rate (req/s)	N	warmup	rate (req/s)	N	warmup
moderate	0.40	1000	100	0.15	500	100
heavy	0.45	1000	100	0.20	500	100

Table 14: W2 on **SGLang** (cell B, $7 \times$ KV pressure): throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. Means across 3 seeds. Both rates run slightly above r_{sat} (errored counts in the single digits), so throughput represents a saturated regime; cache-hit and TTFT deltas remain interpretable.

Load	Metric	LPM+LRU	cLPM+GM+DL+PE
moderate	tput (req/s)	0.289	0.330
	cache hit (%)	62.4	79.4
	TTFT (s)	391.1	178.8
	TPOT (ms)	32.6	31.8
	E2E (s)	419.2	206.8
heavy	tput (req/s)	0.283	0.357
	cache hit (%)	60.0	83.2
	TTFT (s)	543.4	212.2
	TPOT (ms)	31.9	33.4
	E2E (s)	571.5	241.6

Table 15: W2 on **vLLM** (cell B, $7 \times$ KV pressure): throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. All cells run with APC enabled. Means across 3 seeds. errored=0 across all cells.

Load	Metric	FCFS(APC)+LRU	cLPM+GM+DL+PE
moderate	tput (req/s)	0.114	0.114
	cache hit (%)	29.4	33.0
	TTFT (s)	28.4	8.3
	TPOT (ms)	27.4	27.8
	E2E (s)	51.4	31.6
heavy	tput (req/s)	0.143	0.151
	cache hit (%)	29.5	40.1
	TTFT (s)	90.7	24.7
	TPOT (ms)	28.8	31.2
	E2E (s)	114.6	50.5

Phase 1: Prefix trie DFS reordering. PEEK-offline builds a token-level prefix trie over all N submitted requests by inserting the first D tokens of each prompt (default $D=128$); each node records the requests terminating there and the subtree request count. A depth-first traversal that visits children in descending subtree-count order emits a permutation π in which prefix-sharing requests

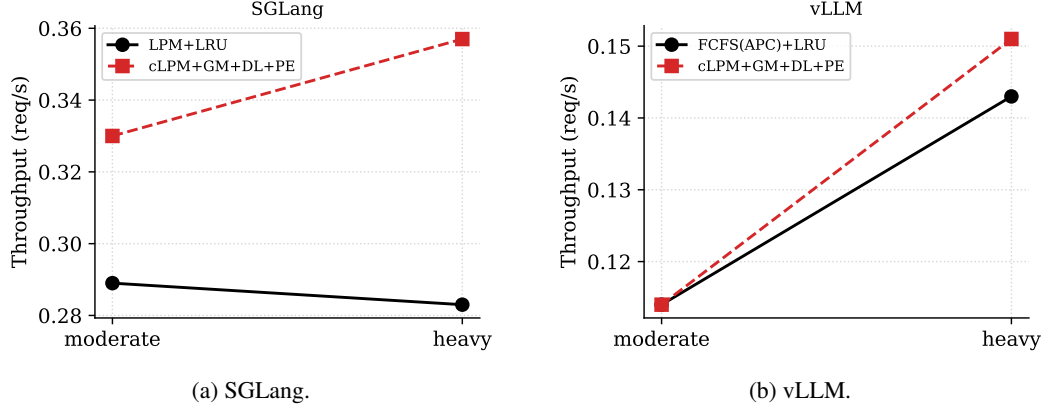


Figure 10: W2 throughput on cell B, both engines and both load levels.

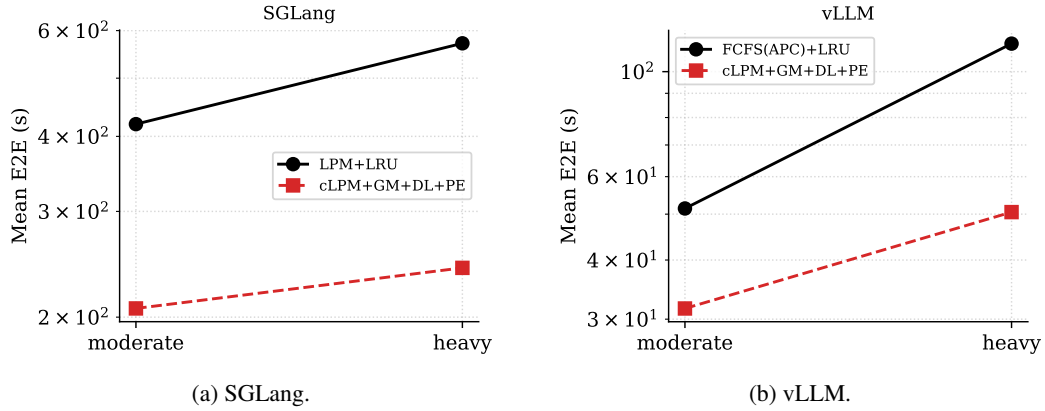


Figure 11: W2 mean E2E latency on cell B (log y-axis), both engines and both load levels.

Table 16: W3 heavy load on **SGLang** (Llama-3.1-70B, TP=2): throughput, cache hit, mean TTFT, mean TPOT, mean E2E at DP=1 (2×H100) and DP=2 (4×H100). Means across 3 seeds.

Cell	Metric	DP=1 (2×H100)		DP=2 (4×H100)	
		LPM+LRU	cLPM+GM+DL+PE	LPM+LRU	cLPM+GM+DL+PE
B (RAG-like)	tput (req/s)	0.464	1.237	1.043	1.689
	cache hit (%)	55.1	98.2	87.8	96.9
	TTFT (s)	324.2	85.3	180.7	91.4
	TPOT (ms)	32.4	32.8	31.9	32.8
	E2E (s)	330.1	91.3	192.6	103.6
C (chat-like)	tput (req/s)	2.896	6.022	7.757	13.634
	cache hit (%)	44.6	84.2	72.9	91.5
	TTFT (s)	46.6	15.7	28.7	10.2
	TPOT (ms)	47.6	44.3	37.0	36.7
	E2E (s)	50.0	18.9	33.4	14.9

are adjacent, even when the engine cache is cold. Construction is $O(N \cdot D)$ and traversal is $O(|T|)$ with $|T| \ll N \cdot D$ under sharing; at $N=5,000$ the entire reorder completes in <50 ms on a single CPU core.

Phase 2: Shadow LRU cache and wave-aware refinement. The Phase 1 ordering is partitioned into *waves* of size W approximating the engine’s running batch. A client-side shadow LRU cache, hashed identically to vLLM/SGLang ($k_i = \text{hash}(k_{i-1}, \text{tuple}(s[iB:(i+1)B]))$, $B=16$), simulates per-block residency in $O(1)$ amortized time and is reconciled against the engine’s reported `cached_tokens` after every completion. A second cache-aware DFS within each wave scores children by

$$\text{key}(v) = \text{cache_frac}(v) \cdot 10^4 - \text{future_refs}(v) + \text{count}(v) \cdot 10^{-3},$$

Table 17: W3 heavy load on **vLLM** (Llama-3.1-70B, TP=2; APC enabled): throughput, cache hit, mean TTFT, mean TPOT, mean E2E at DP=1 and DP=2. Means across 3 seeds.

Cell	Metric	DP=1 (2×H100)		DP=2 (4×H100)	
		FCFS(APC)+LRU	cLPM+GM+DL+PE	FCFS(APC)+LRU	cLPM+GM+DL+PE
B (RAG-like)	tput (req/s)	0.241	1.072	0.478	2.152
	cache hit (%)	37.7	97.1	55.4	97.3
	TTFT (s)	559.6	83.2	384.4	54.3
	TPOT (ms)	36.0	65.9	42.1	93.9
	E2E (s)	572.7	103.4	399.6	83.8
	tput (req/s)	1.798	3.728	4.040	11.440
C (chat-like)	cache hit (%)	37.2	76.5	51.4	92.7
	TTFT (s)	75.8	24.9	60.5	12.3
	TPOT (ms)	32.1	48.1	32.8	43.7
	E2E (s)	79.9	31.0	64.7	17.9

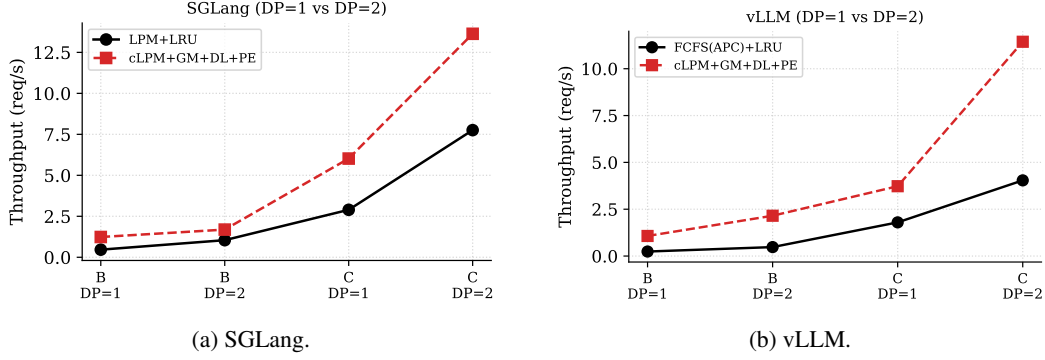


Figure 12: W3 throughput, DP=1 vs DP=2, heavy load.

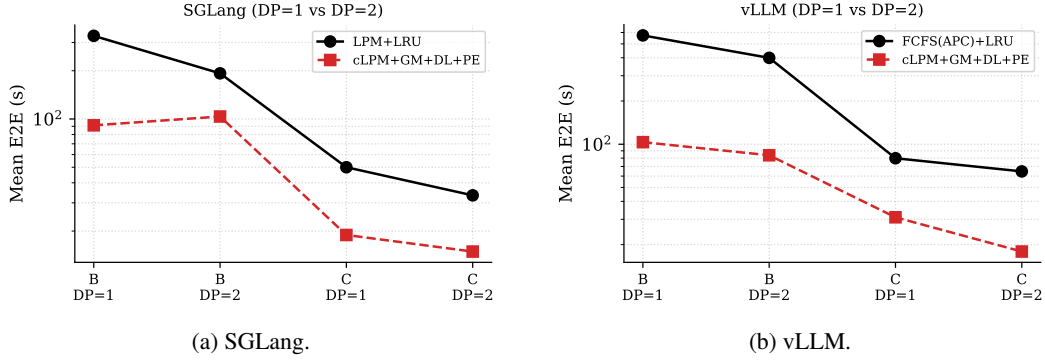


Figure 13: W3 mean E2E latency (log y-axis), DP=1 vs DP=2, heavy load.

Table 18: W4 heavy load on **SGLang**: throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. Means across 3 seeds.

Scenario	Metric	FCFS+LRU	LPM+LRU	cLPM+GM+DL+PE
agentic_only	tput (req/s)	3.58	3.58	3.62
	cache hit (%)	59.5	60.4	60.7
	TTFT (ms)	1175	1129	1185
	TPOT (ms)	65.8	63.9	63.7
	E2E (s)	22.4	22.0	21.8
agentic_shared	tput (req/s)	3.59	3.58	3.62
	cache hit (%)	59.7	60.0	61.0
	TTFT (ms)	1181	1142	1157
	TPOT (ms)	66.3	65.1	62.8
	E2E (s)	22.4	22.2	21.6

Table 19: W4 heavy load on **vLLM**: throughput, cache hit rate, mean TTFT, mean TPOT, mean E2E. All cells run with APC enabled. Means across 3 seeds.

Scenario	Metric	FCFS(APC)+LRU	cLPM+GM+DL+PE
agentic_only	tput (req/s)	3.57	3.58
	cache hit (%)	85.6	87.2
	TTFT (ms)	178	167
	TPOT (ms)	22.8	22.5
	E2E (s)	5.40	5.31
agentic_shared	tput (req/s)	3.54	3.54
	cache hit (%)	91.5	91.5
	TTFT (ms)	171	173
	TPOT (ms)	22.8	22.9
	E2E (s)	4.78	4.79

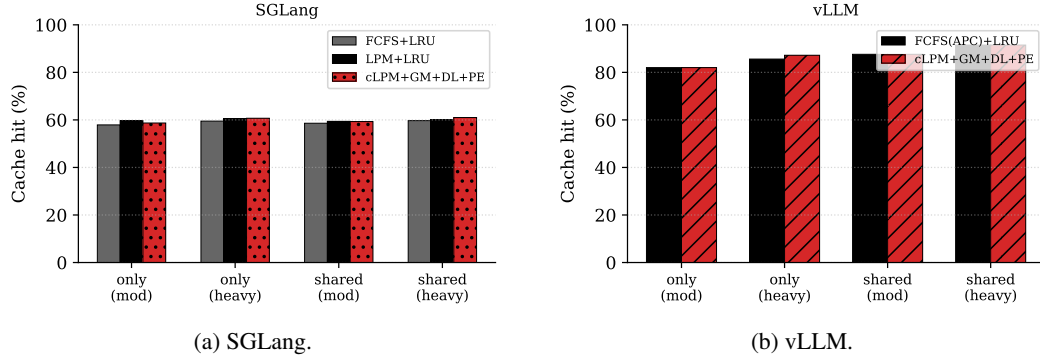


Figure 14: W4 cache hit rate by scenario and load level, side-by-side per engine.

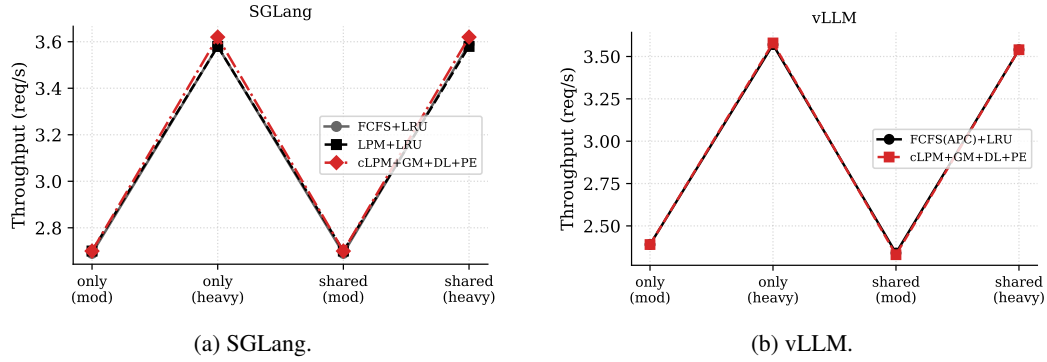


Figure 15: W4 throughput by scenario and load level, side-by-side per engine.

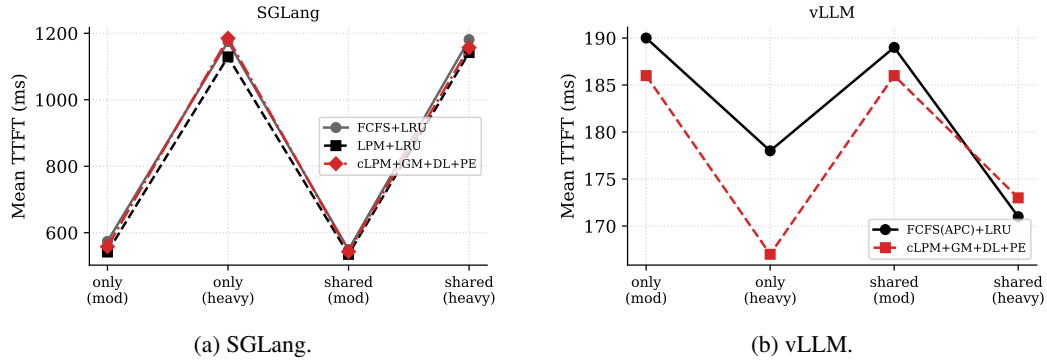


Figure 16: W4 mean TTFT by scenario and load level, side-by-side per engine.

strictly preferring already-cached groups (exploit), deferring groups demanded by later waves (pre-serve), and using subtree count only as a tiebreaker. A queue-aware eviction hook scans the remaining

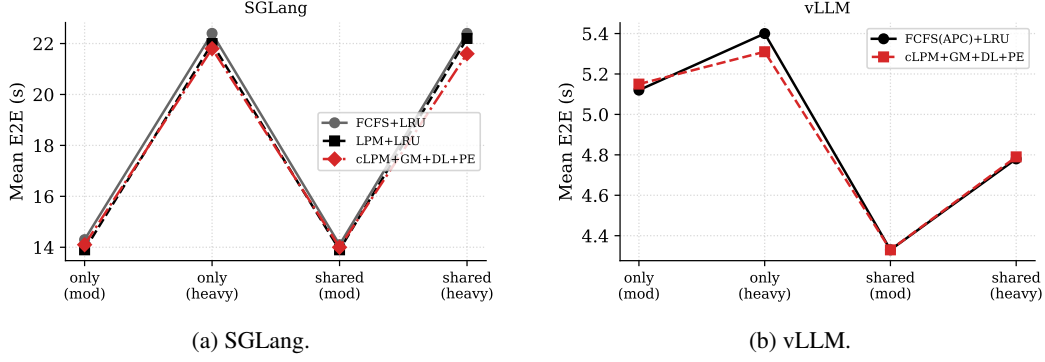


Figure 17: W4 mean E2E latency by scenario and load level, side-by-side per engine.

Table 20: W5 heavy load on **SGLang**: throughput, cache hit, mean TTFT, mean TPOT, mean E2E. Means across available seeds (3 seeds for cell_C_short; cell_C_long: seed 242 only). Cache hit computed from Prometheus snapshots (RadixAttention requires exact-prefix matches; with no shared system prompt, baseline cache hit is near zero).

Cell	Metric	LPM+LRU	cLPM+GM+DL+PE
C_long	tput (req/s)	7.83	8.20
	cache hit (%)	0.7	10.2
	TTFT (s)	21.7	19.4
	TPOT (ms)	52.2	52.7
	E2E (s)	28.3	26.0
C_short	tput (req/s)	21.89	21.68
	cache hit (%)	3.7	3.7
	TTFT (s)	3.6	3.6
	TPOT (ms)	54.6	54.9
	E2E (s)	10.05	10.14

Table 21: W5 on **vLLM**: throughput, cache hit, mean TTFT, mean TPOT, mean E2E. Means across 3 seeds. APC’s block-hash matching gives high reported cache hit on byte-recurring tokens across heterogeneous prompts (a property of the cache layer, not of structural sharing in the workload).

Cell	Metric	FCFS(APC)+LRU	cLPM+GM+DL+PE
C_long	tput (req/s)	18.33	22.27
	cache hit (%)	97.5	98.8
	TTFT (s)	4.25	1.61
	TPOT (ms)	73.6	74.3
	E2E (s)	12.45	9.98
C_short	tput (req/s)	39.63	39.70
	cache hit (%)	96.7	96.7
	TTFT (s)	0.39	0.35
	TPOT (ms)	43.9	44.0
	E2E (s)	5.69	5.66

batch to derive per-block reference counts, evicting unreferenced blocks first and protecting referenced blocks via log-dampened depth-weighted priority to avoid ossification.

Guard conditions. Three cheap pre-checks skip all PEEK work when reordering cannot pay for itself: *duplication* ($>50\%$ exact-duplicate sequences), *coverage* ($<10\%$ of requests share ≥ 32 tokens), and *depth* (average sharing depth < 64 tokens). When any guard fires, PEEK-offline reduces to FCFS at $O(1)$ overhead.

A.3.2 Experiment Results

PEEK-offline is evaluated on two configurations: *Single-GPU* (Qwen2.5-14B-Instruct, $1 \times H100$ 80 GB, FP16) and *Multi-GPU* (Qwen2.5-72B-Instruct, $2 \times H100$, TP=2, FP8), both on SGLang v0.5.9 and vLLM v0.10.x at production-default memory (SGLang `mem_fraction_static=0.80`, vLLM `gpu_memory_utilization=0.90`). Single-GPU uses six workloads (three high-sharing: shared

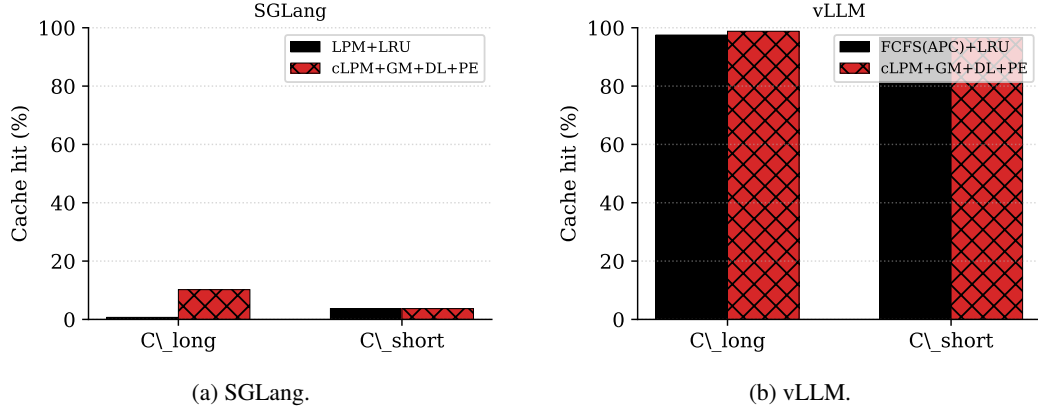


Figure 18: W5 cache hit rate by cell, both engines.

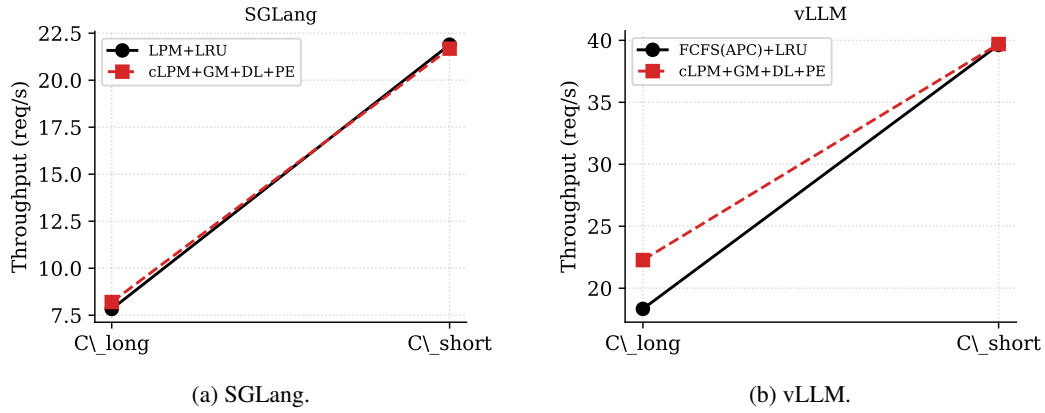


Figure 19: W5 throughput by cell, both engines.

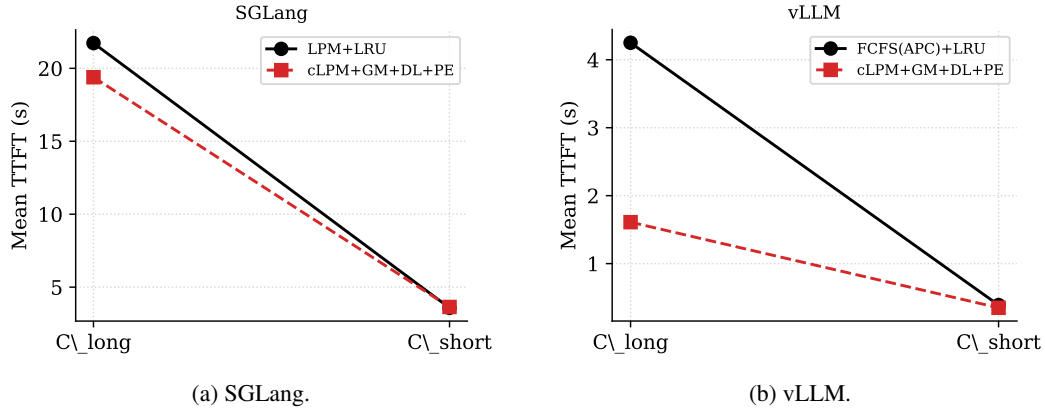


Figure 20: W5 mean TTFT by cell, both engines.

system prompts on LooGLE, mixed traffic, multi-turn ShareGPT chat; three low-sharing: few-shot MMLU, code completion on HumanEval/MBPP, single-turn diverse). Multi-GPU uses six production scenarios (Legal RAG, Long-Doc Summarization, Multi-Agent, Enterprise Search, Mixed Traffic, plus a non-sharing control). Baselines are FCFS, LPM, and DFS-Weight with LRU/LFU on SGLang; FCFS and APC with LRU on vLLM.

Single-GPU. On the three high-sharing workloads, PEEK-offline beats the strongest baseline by **1.1–1.6 $\times$** on SGLang (cache hits 72–84%) and **1.0–1.8 $\times$** on vLLM (71–85%). The largest gains are on shared system prompts where 100 prefix groups arrive randomly interleaved (FCFS achieves only 1% hits); the smallest are on multi-turn chat, where sequential conversation turns supply natural

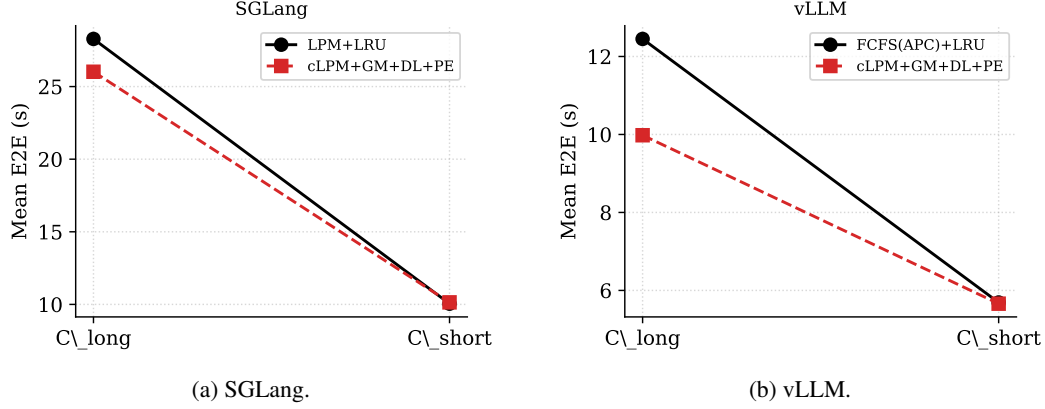


Figure 21: W5 mean E2E latency by cell, both engines.

adjacency (FCFS already at 50% hits). Prefill reduction accounts for 89–97% of total latency improvement; the remainder is decode-side, an indirect benefit of freeing GPU compute sooner. On the three low-sharing workloads, all policies are within 3% of each other—the guards correctly skip reordering. P99/Mean latency ratios stay within 0.05 of the best baseline across all six workloads, confirming no starvation.

Multi-GPU. Scaling to Qwen2.5-72B on $2 \times \text{H100}$, PEEK-offline beats the strongest baseline by **1.6–2.4 $\times$** on SGLang (cache hits 90–96%) across five high-sharing scenarios and by **1.6–8.0 $\times$** on vLLM (90–98%) across four scenarios. The largest gain is on Multi-Agent (16K shared prefixes), the smallest on Mixed Traffic (moderate natural adjacency). Non-sharing controls remain within 5% of baseline.

Component ablation and scaling. A reorder-only variant (DFS without shadow cache or eviction) matches the full system on every workload, while eviction-only delivers no benefit in isolation—DFS reordering supplies the temporal locality that queue-aware eviction then preserves. Sweeps confirm gains scale monotonically with cache pressure: prefix length 512 $\rightarrow$ 4096 grows the gain from 1.0 $\times$ to **3.0–3.8 $\times$** ; group count 20 $\rightarrow$ 200 from $\sim 1 \times$ to **2.2 $\times$** ; concurrency 16 $\rightarrow$ 256 stays in the **1.3–1.8 $\times$** range. Under variable prefix lengths ($\sigma=0.0\text{--}0.6$) PEEK’s hit rate holds at 83–85% while baselines degrade.

Takeaway. Across both single- and multi-GPU offline serving, queue-derived prefix structure is the dominant lever for cache efficiency; the same insight that drives PEEK-online in this paper carries over to the batch regime, where the per-cycle pending tree collapses to a one-shot batch trie but the underlying co-design of prefix-grouped admission and queue-aware eviction is unchanged.